

IA para Desenvolvedores

Como Usar Inteligência Artificial para
Programar Melhor — sem Cair nos Erros
Comuns

Ilvan Joaquim

IA para Desenvolvedores

*Como Usar Inteligencia Artificial para Programar Melhor - sem Cair
nos Erros Comuns*

Autor: Ivan Joaquim

Idioma: pt-BR

Edicao: 1 - 2026

O Erro de 90% ao Usar IA para Programar

1. Introdução: por que 90% cometem esse erro

Era uma terça-feira comum. Um desenvolvedor precisava de uma função simples para validar emails. Pediu ao ChatGPT, copiou o código, fez deploy. Na sexta, o banco de dados estava cheio de registros com emails como "usuario@". Aquela função -- que parecia perfeita -- só verificava se existia um "@" na string. A IA acertou a sintaxe. Errou a lógica. O desenvolvedor não revisou.

Produção quebrou. Já vi esse filme umas cinquenta vezes, em contextos diferentes, com linguagens diferentes, com pessoas diferentes. Sempre o mesmo padrão: a IA entrega algo que parece certo, o desenvolvedor confia, e o erro aparece dias depois num bug report. Esse cenário se repete milhares de vezes todos os dias. Assistentes de IA geram código em segundos -- mas essa velocidade tem um custo

[Diagrama] Mermaid

```
---
title: O Custo da Confianca Cega em IA
---
xychart-beta
  title "Impacto da Configuracao de IA na Taxa de Erro"
  x-axis "Cenario" [ "Sem Regras", "Com 4 Regras", "Com 12 Regras", "Devs que Erraram" ]
  y-axis "Percentual (%)" 0 --> 100
  bar [ 40, 11, 3, 63 ]
  line [ 40, 11, 3, 63 ]
```

Dados do Claude Code Pro Pack e Stack Overflow Survey 2025. A

configuracao de 12 regras reduz a taxa de erro de 40% para 3% -- melhoria de 13.3x.

O problema não é a IA. O problema é **como usamos a IA**. A maioria dos desenvolvedores repete os mesmos 5 erros -- e 90% sequer percebe que os está cometendo.

Na prática: Este capítulo compila fontes oficiais (GitHub, Anthropic, OpenCode), pesquisa acadêmica (arXiv 2512.05239) e benchmarks da indústria para mapear os erros e, mais importante, mostrar como corrigi-los.

Pare e pense: quantas vezes você copiou código de IA sem ler cada linha? Se a resposta for "algumas vezes", este capítulo é para você.

2. Erro 1: Confiar cegamente na saída da IA

O exemplo que deveria ter me alertado

```
[javascript]
// [RUIM] Código gerado por IA -- parece certo, mas está errado
function calculateDiscount(price, coupon) {
  if (coupon === 'SAVE10') {
    return price * 0.9;
  }
  return price; // [RUIM] Esqueceu de validar se price é número
}

calculateDiscount('cem reais', 'SAVE10'); // NaN em produção
```

Atenção: Sempre valide tipos de entrada em funções que processam valores monetários. Um NaN pode custar dinheiro real.

Esse código compila. Roda. Não lança erro. Mas `calculateDiscount('cem reais', 'SAVE10')` retorna NaN. E NaN em produção geralmente vira preço zero, venda de graça, prejuízo.

O problema é que a IA gera respostas com **alta fluência e aparência de confiança**. O código parece correto, compila e muitas vezes até passa em testes simples -- mas pode conter bugs sutis de

Lógica, segurança ou performance.

A IA erra com confiança, não com hesitação." -- TechTudo

Tem um viés psicológico aí: nosso cérebro associa fluência a competência. Quando a IA escreve uma função que "soa bem", a gente relaxa a guarda. Só que a IA não sabe o que está fazendo -- ela está apenas completando padrões estatísticos.

Um survey (arXiv:2512.05239) classifica os bugs encontrados em

[Diagrama] Mermaid

```
%%{init: {"quadrant": {"quadrantPadding": 10, "xAxisLabelPadding": 15, "yAxisLabelPadding": 15}}
quadrantChart
    title Os 4 Tipos de Bug em Código Gerado por IA
    x-axis "Menos Crítico" --> "Mais Crítico"
    y-axis "Menos Frequente" --> "Mais Frequente"
    quadrant-1 "Lógica"
    quadrant-2 "Segurança"
    quadrant-3 "Performance"
    quadrant-4 "Compatibilidade"
    "Validação de email sem @!" [0.3, 0.8]
    "SQL injection" [0.8, 0.6]
    "Loop aninhado N+1" [0.4, 0.5]
    "Import de biblioteca inexistente" [0.2, 0.3]
```

Distribuição dos 4 tipos de bug. Lógica é o mais frequente;

Segurança é o mais crítico.

Consequência

Bugs em produção, vulnerabilidades de segurança e dívida técnica acumulada. Como alerta a documentação oficial do GitHub Copilot, *"Remember that you are in charge, and Copilot is a powerful tool at your service."*

Traduzindo: o responsável é você. A ferramenta é só a ferramenta.

Como corrigir

Passo a passo que uso com meus times para todo código gerado por

IA:

- ****Leia cada linha antes de implementar**** -- se não entendeu, não use

- ****Teste com casos extremos**** -- string vazia, null, negativo, tipos inesperados
- ****Valide dependências sugeridas**** -- a IA pode inventar bibliotecas que não existem
- ****Use type checking**** (TypeScript, mypy, etc.) para pegar erros de tipo

Dica: Se voce nao entendeu o codigo gerado pela IA, nao use. Peca a ela que explique linha por linha.

Para seu contexto: Trate a saída da IA como um **rascunho inicial**, não como produto acabado. A diferença entre um profissional e um amador é que o profissional verifica antes de entregar.

3. Erro 2: Prompts vagos sem contexto

O problema

"Faz aí uma função de login pra mim."
Ja recebi um prompt assim de um colega. A IA devolveu algo em PHP (o projeto era Node.js), com sessão (o projeto usava JWT), e sem tratamento de erro (o projeto tinha padrão `Result<T,E>`). Resultado: três horas perdidas iterando até chegar perto do que precisava. Pedido fraco, resposta fraca em escala industrial.

Por que acontece

A IA opera com base em probabilidades: quanto menos contexto, mais genérica a resposta. É como perguntar "Me recomenda um filme?" para um amigo -- você vai receber uma lista genérica. Agora pergunte "Me recomenda um filme de suspense coreano com menos de 2 horas" e a resposta muda completamente. O mesmo vale para código.

Consequência

Respostas genéricas que não resolvem o problema real. O desenvolvedor perde tempo iterando sobre sugestões irrelevantes, se frustra com a ferramenta e culpa a IA -- quando o problema era o

prompt.

Exemplo concreto

<table>

<tr>

<td width="50%">[BQM]

Qual critério de "melhor"?

- Qual critério de "melhor"?
- Performance? Legibilidade? Segurança?
- Qual o contexto do projeto?

</td>

<td width="50%">

Prompt esotérico [BQM]
Relatore a função handleSubmit no arquivo src/forms.ts para usar async/await com try-catch, mantendo o mesmo comportamento e seguindo o padrão de error handling do resto do projeto."

A IA sabe qual arquivo e função exatos

- O que fazer (refatorar)
- Como fazer (async/await + try-catch)
- Restrições (manter comportamento, seguir padrão existente)

</td>

</tr>

Atencao: Prompt vago nao economiza tempo -- ele terceiriza a decisao para a IA, que vai chutar o que voce nao especificou.

Como corrigir

[Diagrama] Mermaid

graph LR

A["fa:fa-message Contexto"] --> B["fa:fa-bullseye Intencao"]

B --> C["fa:fa-file-lines Formato"]

C --> D["fa:fa-check Resultado"]

A --> A1["Ex: 'No arquivo login.ts, funcao authenticateUser...'"]

B --> B1["Ex: '...precisa validar token JWT antes de consultar o banco'"]

```
C --> C1["Ex: '...retorne Result<T,E>, sem throw, com testes em Vitest'"]
```


style A fill:#E3F2FD,stroke:#1565C0

style B fill:#E3F2FD,stroke:#1565C0

style C fill:#E3F2FD,stroke:#1565C0

style D fill:#E8F5E9,stroke:#2E7D32

```
style A1 fill:#F5F5F5,stroke:#BDBDBD,stroke-dasharray: 3
```

```
style B1 fill:#F5F5F5,stroke:#BDBDBD,stroke-dasharray: 3
```

```
style C1 fill:#F5F5F5,stroke:#BDBDBD,stroke-dasharray: 3
```

Os 3 elementos essenciais de todo prompt. Contexto ancora, Intencao

Dica: Memorize a sigla CIF: Contexto, Intencao, Formato. Todo prompt precisa desses 3 elementos.

Para seu contexto: Antes de escrever um prompt, pergunte-se: "Se eu desse esta instrução para um colega desenvolvedor, ele saberia exatamente o que fazer?" Se a resposta for não, adicione mais contexto.

4. Erro 3: Pular code review e testes

A história que ninguém gosta de contar

Uns anos atrás, um colega meu gerou uma função de pagamento com IA. Testou com o próprio cartão, funcionou. Subiu pra produção. Na primeira cobrança real -- de um cliente, em produção -- a função processou o pagamento duas vezes. A IA tinha um bug de idempotência que passou batido porque ele só testou o caminho feliz. O commit estava com o nome dele. A IA não foi responsabilizada. Ele sim.

Definição

Ignorar as etapas de revisão de código e testes automatizados para código gerado por IA, tratando-o como isento de erros.

Por que acontece

A velocidade da IA cria a ilusão de que o código já passou por um "controle de qualidade implícito". O desenvolvedor assume que, se a IA gerou, está correto.

"Se você copia e cola sem ler, o erro deixa de ser da ferramenta. Passa a ser seu." -- TechTudo

A documentação do Claude Code é categórica:

"Give Claude a check it can run, tests, a build, a screenshot to compare. It's the difference between a session you watch and one you walk away from."

Sem uma verificação executável, "parece pronto" é o único sinal disponível. Você se torna o loop de verificação -- cada erro espera **você** perceber.

Consequência

Dívida técnica acumulada, bugs não detectados, vulnerabilidades de segurança e código de difícil manutenção. O commit vai para o repositório com **seu nome** -- a IA não assume responsabilidade.

Como corrigir

- **Code review obrigatório** -- revise código gerado por IA como revisaria de um colega. Pergunte: "Eu aceitaria isso num PR?"
- **Testes automatizados como verificação** -- antes de aceitar o código, peça para a IA gerar os testes também
- **CI pipeline** -- todo código, inclusive o gerado por IA, deve passar pelos mesmos checks automatizados

***[ATENÇÃO]** Pular code review em código gerado por IA não economiza tempo -- ele terceiriza o risco para você. O bug vai aparecer, a pergunta vai ser "quem autorizou isso?", e a resposta será seu nome no commit.*

5. Erro 4: Ignorar configuração do projeto (AGENTS.md / CLAUDE.md)

O dado que me convenceu

Quando vi a pesquisa do Claude Code Pro Pack, parei tudo que estava

Configuração	Taxa de erro	Melhoria
-----	:-----:	:-----:
Sem regras	~40%	--
Com 4 regras básicas	~11%	~3,6x

Com 12 regras (pro pack)

O Erro de 90% ao Usar IA para Programar | IA para Desenvolvedores

~3%

~13,3x

Quarenta por cento de erro sem configuração. Três por cento com 12 regras. A diferença não é a IA -- é o contexto que você dá pra ela.

Definição

Não configurar arquivos de instrução persistente para os agentes de IA, deixando-os operar sem contexto do projeto.

Por que acontece

Arquivos como `AGENTS.md` (OpenCode) e `CLAUDE.md` (Claude Code) funcionam como a memória de longo prazo do agente. Eles contêm regras, padrões e convenções do projeto. Sem eles, a IA opera com conhecimento genérico. Ela não sabe se o projeto usa React 18 ou Vue 3. Não sabe se prefere `const` ou `function`. Não sabe se os testes são com Vitest ou Jest. Ela chuta.

"5 minutos de configuração economizam horas de retrabalho."

-- OpenCode Community

Consequência

Comportamento inconsistente do agente -- o código gerado muda de estilo a cada interação, ignora padrões do projeto e força retrabalho manual.

Como corrigir

Crie um arquivo de instrução na raiz do projeto. A estrutura mínima

```
[markdown]
# Agente: Front-end

## Stack
React 18 + TypeScript + Tailwind CSS

## Regras
- Use arrow functions para componentes
- Prefira `const` sobre `let`
- Testes com Vitest, não Jest
- Erros devem usar `Result<T, E>` (never throw)
```

- Nomes de arquivo em kebab-case

- Componentes em `src/components/`, páginas em `src/pages/`

Para seu contexto: Invista 5 minutos agora para criar o `AGENTS.md`. É o investimento com maior retorno por minuto no uso de IA para programar. A cada novo projeto, comece por ele.

Para e pense: seu projeto atual tem um arquivo de instruções para a IA? Se não, esse é o erro número 1 que você está cometendo sem perceber.

6. Erro 5: Tratar IA como resposta final

O impulso do "já está bom"

É o mesmo impulso de mandar um email sem reler -- a gratificação imediata de "pronto" supera a disciplina de refinar. A IA entrega respostas completas e aparentemente prontas, e a gente aceita como definitivas.

Definição

Usar a primeira resposta da IA como solução definitiva, sem refinamento ou iteração.

Por que acontece

A IA entrega respostas completas e aparentemente prontas. O desenvolvedor assume que a primeira tentativa é a melhor e encerra o ciclo ali.

A primeira resposta raramente é a melhor. Ela é o ponto de partida, não o produto final." -- TechTudo

Consequência

Resultados superficiais. O código funciona no caminho feliz, mas quebra nos casos extremos. O desenvolvedor perde a oportunidade de refinar, corrigir e adaptar a solução ao contexto real. Pior: nunca sabe o que perdeu.

Exemplo concreto


```
[javascript]
// [RUIM] Primeira resposta (rascunho)
function validateEmail(email) {
  return email.includes('@');
  // [RUIM] Aceita "@", "a@", "a@b"
}

// [BOM] Versão refinada com feedback
function validateEmail(email) {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
  // [BOM] Rejeita "@", "a@", "a@b", aceita "user@domain.com"
}
```

Como corrigir

Trate IA como **processo iterativo**, não como resposta final:

- ****Revise e identifique pontos de melhoria**** -- o que está faltando?
- ****Refine com feedback direcionado**** -- seja específico sobre o que mudar
- ****Repita até atender aos critérios de qualidade**** -- quando passar nos testes e na revisão

Para seu contexto: Em vez de reescrever o prompt do zero, **itere através de feedback**. Diga à IA especificamente o que ajustar:

"Mude a nomenclatura para camelCase", "Extraia essa lógica para um hook separado" ou "Adicione tratamento para o caso de lista vazia."

A pesquisa SPER Institute confirma: desenvolvedores que iteram com feedback estruturado reduzem em **~35% as iterações**

necessárias em comparação com quem reescreve prompts do zero.

7. Conclusão: como usar IA corretamente

Usar IA para programar não é sobre aceitar código -- é sobre **colaborar**

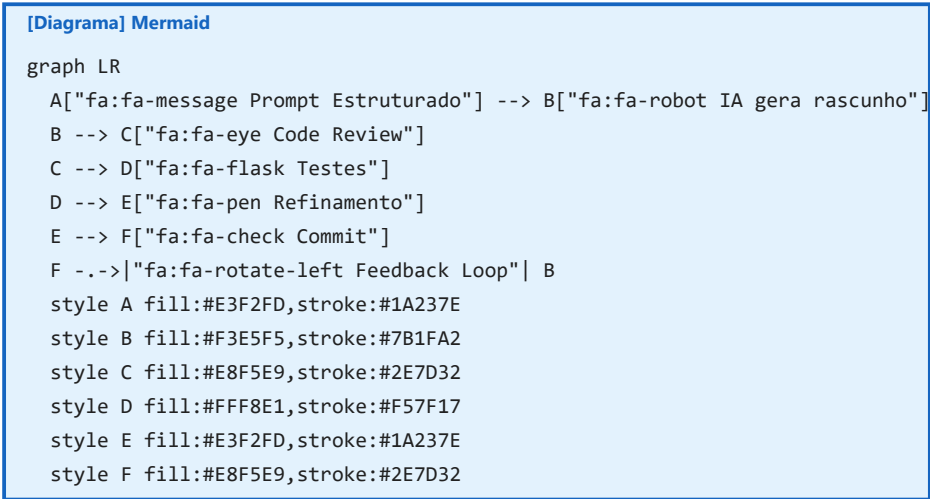
ação inteligente. A IA é uma ferramenta poderosa, mas sem direção, validação e contexto, ela produz resultados medíocres.

Os 5 mandamentos do uso correto de IA

#	Mandamento	Por quê
1	**Valide** toda saída da IA	IA erra com confiança, não com hesitação
2	**Estruture** prompts com contexto	Pedido fraco [RUIM] resposta fraca
3	**Revise e teste** como qualquer código	Seu nome está no commit
4	**Configure** o projeto para a IA	5 minutos economizam horas
5	**Itere** , não aceite a primeira resposta	Refinamento separa o mediano do excelente

Na prática: Desenvolvedores que aplicam essas práticas reduzem em **~35% as iterações necessárias** (SFEIR Institute) e produzem código com **~97% de acerto** (Claude Code Pro Pack).

Diagrama conceitual: Fluxo ideal de uso de IA



Fluxo iterativo: a IA gera um rascunho, voce revisa, testa, refina e so

entao faz o commit. O feedback loop entre Commit e IA gera rascunho representa o refinamento continuo.

Analogia principal

Usar IA para programar é como ter um **estagiário brilhante, mas inexperiente**. Ele trabalha rápido, escreve bem, mas precisa de supervisão, contexto e revisão constantes. Confiar cegamente nele é negligência; ignorá-lo é desperdício. O profissional sábio sabe

Recapitulando: Voce aprendeu os 5 erros: Confiar cegamente sem validar, Prompts vagos sem contexto, Pular code review, Ignorar configuracao do projeto, Tratar IA como resposta final.

Exercício prático

Título: Diagnosticando um prompt ruim
Duração: 5 minutos
Instruções: Leia o prompt abaixo:

- > "Faz aí uma função de login pra mim"
1. Liste 3 problemas com este prompt (erro 2 -- prompt vago)
- Reescreva o prompt seguindo a estrutura ****Contexto + Intenção + Formato Esperado****
 - Se possível, execute o prompt reescrito em um assistente de IA e compare a qualidade da resposta

- Critérios de sucesso:**
- Identificou a falta de: linguagem/framework, definição de sucesso, tratamento de erros
 - Novo prompt inclui pelo menos: stack tecnológica, regras de negócio, formato da resposta esperada

Problema	Explicação
Falta linguagem/framework	"Faz aí" não diz se é Node, Python, PHP, etc.
Falta definição de sucesso	O que é "login"? JWT? Sessão? OAuth?

Falta tratamento de erros

E se o usuário não existe? Senha errada?
Taxa de limite?

Prompt reescrito (exemplo):
Crie uma função de login em Node.js com Express. O usuário envia email e senha no corpo da requisição. Valide o email com regex, compare a senha com bcrypt, e retorne um token JWT com expiração de 24h. Se o email não existir ou a senha estiver errada, retorne 401 com mensagem clara. Use `async/await` com `try-catch`."

Desafio final

Título: Auditoria de código gerado por IA
Duração: 10 minutos
Cenário: Você recebeu um Pull Request com código 100% gerado por IA. O desenvolvedor apenas copiou e colou sem revisar.
Tarefa: Analise o trecho abaixo, identifique **todos os erros** e

```
[python]
import os
import hashlib

def hash_password(password):
    # Gerar hash simples
    return hashlib.md5(password.encode()).hexdigest()

def save_user(username, password):
    hashed = hash_password(password)
    query = f"INSERT INTO users (username, password) VALUES ('{username}', '{hashed}')"
    os.system(f"mysql -e \"{query}\"")
```

- O que procurar:**
- 1. **Erro 1:** confiar cegamente -- código tem vulnerabilidades graves
 - 2. **Erro 3:** pular code review -- falta de testes e validação
 - 3. **Erro 5:** tratar como resposta final -- código inseguro em produção

Erro identificado	Por que é grave	Correção
MD5 para senhas	MD5 é instantaneamente	

quebrável com ataques de
dicionário

Use ``bcrypt`` ou ``argon2``

SQL injection

String	formatada
diretamente	permite
injeção de comandos SQL	

Use prepared statements /
ORM

```
`os.system`
```

Expõe o shell a comandos
maliciosos; desnecessário

```
Use biblioteca MySQL com  
parâmetros  
(`mysql-connector-python  
`)
```

Falta validação de entrada

``username` pode conter
caracteres maliciosos`

Valide e sanitize entradas


```
[python]
import os
import bcrypt
import mysql.connector

def hash_password(password):
    return bcrypt.hashpw(password.encode(), bcrypt.gensalt())

def save_user(username, password):
    if not username or not password:
        raise ValueError("Campos obrigatórios")

    hashed = hash_password(password)
    conn = mysql.connector.connect(
        host=os.getenv("DB_HOST", "localhost"),
        user=os.getenv("DB_USER", "app"),
        password=os.getenv("DB_PASSWORD"),
        database=os.getenv("DB_NAME", "appdb")
    )
    cursor = conn.cursor()
    cursor.execute(
        "INSERT INTO users (username, password) VALUES (%s, %s)",
        (username, hashed)
    )
    conn.commit()
    cursor.close()
    conn.close()
```

Leituras complementares

- GitHub Copilot Best Practices -- <https://docs.github.com/en/copilot/get-started/best-practices>
- Claude Code Best Practices -- <https://code.claude.com/docs/en/best-practices>
- OpenCode Official Docs -- <https://opencode.ai/>
- "A Survey of Bugs in AI-Generated Code" (arXiv 2512.05239) --

<https://arxiv.org/abs/2512.05239>

- 5 erros ao usar IA que sabotam suas respostas (TechTudo) --
<https://www.techtudo.com.br/listas/2026/04/5-erros-ao-usar-ia-que-sabotam-suas-respostas-e-como-evita-los-edsoftwares.g.html>
- CLAUDE.md Rules: How to Cut AI Coding Mistakes from ~40%
to ~3% (DEV.to) --
<https://dev.to/rams901/claudemd-rules-how-to-cut-ai-coding-mistakes-from-40-to-3-in-2026-2j70>
- 10 Most Common Mistakes Using AI Coding Tools (Ryz Labs) --
<https://www.ryzlabs.com/>

Agentes de IA na Prática

Construindo seu exército de agentes.

1. Por que agentes especializados?

Quando montei meu primeiro time de agentes, comecei com um único agente genérico -- "você é um desenvolvedor full stack". O resultado era mediano em tudo: o código frontend funcionava, o backend rodava, mas a qualidade era inconsistente. Um dia ele gerava um componente React lindo, no outro esquecia validação de entrada numa rota crítica.

A lição foi clara: um agente genérico produz resultados **medianos em todas as áreas**. Um ecossistema de agentes especializados produz **resultados excelentes em cada área**.

O problema do agente genérico

[Diagrama] Mermaid

```
graph TB
    subgraph AG["Agente Genérico"]
        direction TB
        G1["Frontend: 2/5<br/>React basico"] --> G2["Backend: 3/5<br/>API basica"]
        G2 --> G3["Seguranca: 1/5<br/>Esquece CSRF"]
        G3 --> G4["Banco: 2/5<br/>Faz N+1"]
        G4 --> G5["DevOps: 1/5<br/>Sem multi-stage"]
    end

    subgraph AE["Agente Frontend Especializado"]
        direction TB
        E1["Performance: 5/5<br/>Lazy loading, Suspense"] --> E2["Acessibilidade: 5/5<br/>A11y"]
        E2 --> E3["SEO: 5/5<br/>Metadata API"]
        E3 --> E4["TypeScript: 5/5<br/>Strict mode"]
    end

    AG -->| "Resultado: divida tecnica" | RES1["Codigo 'funciona'<br/>mas fragil"]
    AE -->| "Resultado: qualidade" | RES2["Codigo de producao<br/>pronto para review"]
    style AG fill:#FFEBEE,stroke:#C62828
```

```
style AE fill:#E8F5E9,stroke:#2E7D32
```

`style G1 fill:#FFCDD2,stroke:#C62828`

style G2 fill:#FFCDD2,stroke:#C62828

style G3 fill:#FFCDD2,stroke:#C62828

style G4 fill:#FFCDD2,stroke:#C62828


```
style G5 fill:#FFCDD2,stroke:#C62828
```

`style E1 fill:#C8E6C9,stroke:#2E7D32`

style E2 fill:#C8E6C9,stroke:#2E7D32

style E3 fill:#C8E6C9,stroke:#2E7D32

style E4 fill:#C8E6C9,stroke:#2E7D32

```
style RES1 fill:#FFEBEE,stroke:#C62828
```

```
style RES2 fill:#E8F5E9,stroke:#2E7D32
```

Comparacao visual: um agente generico sabe de tudo um pouco mas e mediano em tudo. Um ecossistema de agentes especializados

produz resultados excelentes em cada area.

2. Anatomia de um agente

Definicao: Um agente de IA e um prompt especializado com identidade, conhecimento, processo e criterios de qualidade definidos. Diferente de um prompt avulso, um agente tem estrutura, limites e reutilizacao.

Cada agente da nossa biblioteca segue a mesma estrutura. Na prática, o que funciona é começar com 3 agentes e expandir conforme surgem

```
1 agente/  
  |— README.md           # Identidade: objetivo, responsabilidades, stack  
  |— workflow.md         # Processo: fluxo de trabalho passo a passo  
  |— checklist.md        # Qualidade: o que validar antes de entregar  
  |— prompts/            # Instruções: templates de prompt para tarefas comuns  
  |   |— prompt-tarefa-1.md  
  |   |— prompt-tarefa-2.md
```

Componentes de um agente eficaz

[Diagrama] Mermaid

```
graph TB  
  subgraph ID["IDENTIDADE"]  
    ID1["Quem e este agente?"]  
    ID2["O que sabe fazer?"]  
    ID3["O que NAO faz?"]  
  end  
  subgraph CN["CONHECIMENTO"]  
    CN1["Stack tecnologica"]  
    CN2["Padroes e praticas"]  
    CN3["Referencias externas"]  
  end  
  subgraph PR["PROCESSO"]  
    PR1["Fluxo de trabalho"]  
    PR2["Entrada -> Transformacao -> Saida"]  
  end  
  subgraph QL["QUALIDADE"]
```


QL1["Checklist de validacao"]

QL2["Critérios de aceite"]

QL3["Anti-padroes a evitar"]

end

```
subgraph CM["COMUNICACAO"]
```

```
CM1["Formato de entrada"]
```

```
CM2["Formato de saída"]
```

CM3["Como reportar problemas"]

end

ID --> PR

CN --> PR

PR --> QL

QL --> CM

style ID fill:#E3F2FD,stroke:#1565C0

style CN fill:#E8F5E9,stroke:#2E7D32

```
style PR fill:#FFF8E1,stroke:#F57F17
```


`style QL fill:#FFEBEE,stroke:#C62828`

style CM fill:#F3E5F5,stroke:#7B1FA2

Os 5 componentes essenciais de um agente de IA eficaz. Cada componente tem função específica na definição do comportamento

Dica: Comece com 3 agentes: Code Reviewer, Test Writer e Doc Generator. Expanda conforme necessidade.

3. Os 17 agentes da formação

Agentes de Produto

Agente	Função	Stack/Conhecimento
Product Discovery	Transformar problemas em requisitos	User Stories, RICE, MoSCoW, BDD
UX Research	Validar hipóteses com usuários	Entrevistas, testes de usabilidade, personas

Agentes de Design

Agente	Função	Stack/Conhecimento
UX Designer	Projetar experiência do usuário	User flows, wireframes, acessibilidade (WCAG)
UI Designer	Projetar interface visual	Design System, tokens, dark mode, responsividade

Agentes de Arquitetura

Agente	Função	Stack/Conhecimento
Enterprise Architect	Decisões arquiteturais	Clean Arch, DDD, ADRs, C4 Model
Database Architect	Modelagem de dados	PostgreSQL, índices, partições, migrações

Agentes de Desenvolvimento

Agente	Função	Stack/Conhecimento
Backend Expert	APIs e regras de domínio	NestJS, Prisma, REST, GraphQL, Zod
Frontend Expert	Interfaces e componentes	
		Next.js 14, RSC, Tailwind, shadcn/ui, TanStack Query
Prisma Expert	Schema e queries	Prisma ORM, migrations, N+1 prevention, soft delete

Agentes de Infraestrutura

Agente	Função	Stack/Conhecimento
DevOps Expert	Docker, CI/CD, deploy	Docker multi-stage, GitHub Actions, health checks
Security Expert	Proteção do sistema	
		OWASP Top 10, JWT, bcrypt, rate limiting, CSP

Agentes de Qualidade

Agente	Função	Stack/Conhecimento
QA Expert	Testes automatizados	Jest, Playwright, Testing Library, cobertura >80%
Performance Expert	Otimização	
		Core Web Vitals, caching, bundle analysis, load testing

Agentes de Governança

Agente

Função

Stack/Conhecimento

****Auditor****

16 tipos de auditoria

Score (0-10), (Blocker a Minor), planos de ação

****Documentation****

Documentação técnica

ADRs, README, Swagger,
CHANGELOG,
CONTRIBUTING

****Refactoring****

Refatoração guiada

Code smells, patterns,
TypeScript strict, preservar
comportamento

Nem todo projeto precisa de todos os 17 agentes. Selecione os que resolvem seus

Nem todo projeto precisa de todos os 17 agentes. Selecione os que resolvem seus problemas mais frequentes.

4. Como criar um novo agente

Passo a passo

- ****Definir o DOMÍNIO do agente**** -- Qual área ele cobre? (ex: "segurança de aplicações web")
- ****Definir o CONHECIMENTO BASE**** -- Quais tecnologias, padrões e boas práticas ele domina?
- ****Definir RESPONSABILIDADES**** -- O que ele faz? O que ele NAO faz? (limites são importantes)
- ****Definir o PROCESSO**** -- Qual o fluxo de trabalho? O que recebe na entrada? O que entrega?
- ****Criar CHECKLIST DE QUALIDADE**** -- O que validar antes de considerar o trabalho concluído?
- ****Criar TEMPLATES DE PROMPT**** -- Prompts reutilizáveis para as tarefas mais comuns
- ****TESTAR com um caso real**** -- Executar o agente, revisar o output, ajustar

Exemplo: Criando o Security Expert Agent

Domínio: Segurança de aplicações web

Conhecimento base:

1. OWASP 10 p10 (2021)
2. JWT, OAuth2, MFA
3. bcrypt, Helmet, CORS, CSP
4. NestJS Guards, CASL (autorização)

Responsabilidades:

• Implementar autenticação e autorização

- Prevenir SQL injection, XSS, CSRF

- Configurar rate limiting
- Gerenciar segredos

Limites: Não define arquitetura geral

- Não implementa lógica de negócio
- Não gerencia infraestrutura

```
- [ ] Senhas com hash bcrypt/argon2
- [ ] JWT com expiração curta + refresh token
- [ ] Rate limiting no login
- [ ] CSP header configurado
- [ ] Helmet.js ativado
- [ ] Input validation em todos os endpoints
- [ ] SQL injection prevenido (ORM)
```

5. Como combinar agentes em pipeline

O verdadeiro poder está em **compor** agentes em sequência. Já vi times com 50 agentes que não serviam pra nada porque faltava o básico: definir claramente a entrada e saída de cada um. Sem isso, o pipeline vira bagunça.

Pipeline de features

[Diagrama] Mermaid

```
graph TB
    PD["fa:fa-chart-line Product Discovery"] --> UX["fa:fa-user-gear UX Designer"]
    UX --> UI["fa:fa-pen UI Designer"]
    EA["fa:fa-sitemap Enterprise Architect"] -.-> PD
    EA -.-> UX
    EA -.-> UI
    subgraph IMPL["Implementacao"]
        BE["fa:fa-server Backend Expert"]
        FE["fa:fa-code Frontend Expert"]
        PE["fa:fa-database Prisma Expert"]
        SE["fa:fa-lock Security Expert"]
    end
```

end

UI --> BE

UI --> FE

UI --> PE

UI --> SE

BE --> QA["fa:fa-check-double QA Expert"]

FE --> QA

PE --> QA

SE --> QA

QA --> DO["fa:fa-rocket DevOps Expert"]

```
DO --> AUD["fa:fa-eye Auditor Agent"]
```

AUD --> DOCS["fa:fa-book-open Documentation Agent"]

style PD fill:#E3F2FD,stroke:#1A237E


```
style UX fill:#E3F2FD,stroke:#1A237E
```

```
style UI fill:#E3F2FD,stroke:#1A237E
```

```
style EA fill:#F3E5F5,stroke:#1A237E
```

```
style QA fill:#FFF8E1,stroke:#1A237E
```

style DO fill:#E8F5E9,stroke:#1A237E

```
style DOCS fill:#E8F5E9,stroke:#1A237E
```

Pipeline completo de desenvolvimento de features. O Enterprise Architect fornece diretrizes arquiteturais para os passos iniciais. QA valida antes do deploy.

Pipeline de auditoria

[Diagrama] Mermaid

```
graph TB
    FI["fa:fa-code Feature Implementada"] --> SA["fa:fa-lock Security Auditor"]
    SA -->| "Blocker/Critical" | BE["fa:fa-server Backend Expert<br/>corrige"]
    SA -->|"OK"| AA["fa:fa-sitemap Architecture Auditor"]
    AA -->|"Problema"| EA["fa:fa-sitemap Enterprise Architect<br/>revisa"]
    AA -->|"OK"| PA["fa:fa-gauge-high Performance Auditor"]
    PA -->|"Lento"| PX["fa:fa-gauge-high Performance Expert<br/>otimiza"]
    PA -->|"OK"| CQA["fa:fa-check-double Code Quality Auditor"]
    CQA -->|"Abaixo padrao"| RA["fa:fa-rotate Refactoring Agent"]
    CQA -->|"OK"| RPT["fa:fa-file-lines Relatorio Consolidado"]
    style FI fill:#E8F5E9,stroke:#2E7D32
    style SA fill:#FFEBEE,stroke:#C62828
    style AA fill:#FFF8E1,stroke:#F57F17
    style PA fill:#FFF8E1,stroke:#F57F17
    style CQA fill:#E3F2FD,stroke:#1A237E
    style RPT fill:#E8F5E9,stroke:#2E7D32
```

Pipeline de auditoria em 4 etapas. Cada auditor especializado verifica um aspecto diferente. Se encontrar problemas, o agente especialista correspondente corrige.

Pipeline de onboarding

[Diagrama] Mermaid

```
graph TB
    PO["fa:fa-user PO descreve problema"] --> PDA["fa:fa-chart-line Product Discovery"]
    PDA --> UXR["fa:fa-user-gear UX Researcher"]
    UXR --> UXD["fa:fa-pen UX Designer"]
    UXD --> UID["fa:fa-pen UI Designer"]
    UID --> EA["fa:fa-sitemap Enterprise Architect"]
    EA --> IMPL["fa:fa-server Backend + Frontend + Prisma + Security"]
    IMPL --> QA["fa:fa-check-double QA Agent"]
    QA --> DO["fa:fa-rocket DevOps Agent"]
```

```
DO --> AUD["fa:fa-eye Auditor Agent"]
```


style PO fill:#E3F2FD,stroke:#1A237E

```
style PDA fill:#E3F2FD,stroke:#1A237E
```

```
style UXR fill:#F3E5F5,stroke:#7B1FA2
```

```
style UXD fill:#F3E5F5,stroke:#7B1FA2
```

`style UID fill:#F3E5F5,stroke:#7B1FA2`

```
style EA fill:#FFF8E1,stroke:#F57F17
```

```
style IMPL fill:#E8F5E9,stroke:#2E7D32
```

```
style QA fill:#FFF8E1,stroke:#F57F17
```


style DO fill:#E8F5E9,stroke:#2E7D32

```
style AUD fill:#FFEBEE,stroke:#C62828
```

Pipeline de onboarding completo: do problema em linguagem natural ao score final do auditor. Cada estagio produz um artefato que alimenta o proximo.

6. Integração com OpenCode

Configuração de agentes no opencode.json

```
[json]
{
  "agents": {
    "frontend-expert": {
      "prompt": ".opencode/agents/frontend-expert.md",
      "permissions": {
        "bash": true,
        "read": true,
        "edit": true,
        "glob": true,
        "grep": true
      }
    },
    "auditor": {
      "prompt": ".opencode/agents/auditor.md",
      "permissions": {
        "bash": true,
        "read": true,
        "edit": false,
        "glob": true,
        "grep": true
      }
    }
  }
}
```

Como invocar um agente

```
@frontend-expert Crie um componente de tabela com:
- Server Component
```

- Suporte a sort e filter

- Paginação

- Loading state com Suspense

Como fazer um agente revisar outro

```
@auditor Revise a segurança deste endpoint.  
  
[endpoint code]
```

7. Boas práticas na criação de agentes

Faça

- ****Seja específico**** -- "Crie componente com Server Component" não "Faça um componente bonito"
- ****Defina limites**** -- "Este agente NAO implementa regras de domínio"
- ****Forneça exemplos**** -- "Siga este padrão: [exemplo]"
- ****Crie checklists**** -- "Antes de entregar, verifique: [itens]"
- ****Itere**** -- Ajuste os prompts baseado nos resultados

Não faça

- ****Não misture domínios**** -- Um agente de backend não deve ter responsabilidades de frontend
- ****Não seja vago**** -- "Seja criativo" não é uma instrução útil
- ****Não ignore limites**** -- Se o agente não tem conhecimento, ele vai alucinar
- ****Não pule a revisão**** -- Sempre revise o output, especialmente no início

Padrão de prompt eficaz

```
Ruim:  
"Crie uma API de usuários."
```


Bom:

"Crie uma API REST de usuários com NestJS seguindo Clean Architecture.

Requisitos:

- POST /users (criar)

- GET /users (listar com paginação)

- GET /users/:id (detalhe)

- PUT /users/:id (atualizar)

- DELETE /users/:id (soft delete)

Validações:

- Email: formato válido, único

- Nome: 3-100 caracteres

- Senha: mínimo 8 caracteres, 1 número, 1 maiúscula

Regras:

- Usar Prisma para persistência

- Zod para validação

- Swagger para documentação

- Tratamento de erros com NestJS exception filters"

8. O futuro: agentes que criam agentes

O próximo passo natural: um **Meta-Agent** que cria agentes especializados sob demanda. Hoje já tenho uma visão clara de como isso funciona na prática -- o que falta é automatizar a orquestração.

Como funcionaria

- **Meta-Agent analisa o problema**
- **Identifica os domínios necessários**
- **Cria um agente especializado para cada domínio**
- **Define os limites e responsabilidades**
- **Testa e ajusta cada agente**
- **Combina em pipeline**
- **Executa o pipeline completo**

O que já temos

- A estrutura padronizada de agentes (README, workflow, checklist, prompts)
- 17 agentes especializados pré-construídos
- Pipelines documentados para features, auditoria e onboarding
- Integração com OpenCode

O que podemos construir

- **Agente Orquestrador** -- recebe um problema e coordena os agentes certos
- **Agente de Onboarding** -- configura o workspace com todos os agentes necessários
- **Agente de Métricas** -- monitora a efetividade de cada agente e sugere ajustes

Exercício prático

Título: Crie seu primeiro agente especializado
Cenário: Você está liderando um projeto Next.js com Prisma e precisa de um agente para revisar a qualidade do código antes dos PRs.

Tarefa: Estruture os 5 componentes do agente Code Reviewer:
Identidade -- Quem é? O que sabe? O que NÃO faz?

- **Conhecimento** -- Stack, padrões, referências?
- **Processo** -- Fluxo de entrada/saída?
- **Qualidade** -- Checklist de validação?
- **Comunicação** -- Formato do output?

Componente	Conteúdo esperado
Identidade	"Code Reviewer -- especializado em revisar código TypeScript/React/Next.js. NÃO modifica o código."
Conhecimento	ESLint, Prettier, React best practices, TypeScript strict, Prisma patterns
Processo	Entrada: PR diff -> Análise: tipos, performance, segurança -> Saída: relatório com issues
Qualidade	Verificar: tipos corretos, error handling, null safety, imports organizados
Comunicação	Issues priorizadas (Blocker/Critical/Major/Minor) com localização e sugestão

Recapitulando: Agentes especializados > genericos. Use a estrutura padrao (README, workflow, checklist, prompts). Combine em pipelines. O futuro e o Meta-Agent que cria agentes sob demanda.

Automação do Ciclo de Desenvolvimento com IA

1. O que é Automação

Automação é a substituição de processos manuais repetitivos por scripts, pipelines e ferramentas que executam essas tarefas de forma confiável, auditável e escalável. Quando comecei a automatizar pipelines, o maior choque não foi técnico -- foi perceber o tempo que eu perdia em tarefas que um script podia fazer em segundos. Deploy manual de 40 minutos virou pipeline de 3 minutos. Teste que eu rodava esquecendo de rodar virou gate obrigatório.

Por que automatizar?

Motivo	Impacto
Reduzir erro humano	O maior causador de incidentes em produção ainda é o operador humano
Velocidade	Máquinas executam em segundos o que levaria horas manualmente
Reprodutibilidade	O mesmo pipeline executa exatamente da mesma forma todas as vezes
Auditoria	Logs e artefatos gerados automaticamente servem como evidência
Escala	O que funciona para 1 deploy falha para 100 deploys manuais

Onde automatizar

- Build e compilação

- Testes (unitários, integração, e2e)
- Linting e formatação de código
- Verificação de segurança
- Deploy em ambientes
- Migrações de banco de dados
- Geração de changelog e release
- Monitoramento e alertas
- Criação e destruição de ambientes

Custo vs Benefício

A regra prática: **automatize tudo que for executado mais de 2**

Dica: Se voce executou a mesma tarefa manualmente 3 vezes, ja deveria ter

$$\text{Custo de automatizar} = (\text{tempo para criar} + \text{tempo para manter}) \times \text{custo-hora}$$
$$\text{Benefício} = (\text{tempo economizado por execução} \times \text{frequência} \times \text{horizonte}) - \text{custo}$$

Se o benefício for positivo em 6 meses, vale a pena automatizar.

2. CI/CD -- Pipelines de Integração e Deploy Contínuo

Das pipelines que já construí, a que mais me orgulha foi a primeira que substituiu um deploy manual que levava 40 minutos por um processo de 3 minutos. CI/CD não é luxo -- é o mínimo para um time

Definicao: CI/CD: Continuous Integration (integracao continua de codigo) + Continuous Delivery/Deployment (entrega/deploy contínuo). E a espinha dorsal da

[Diagrama] Mermaid

```
graph LR
  GIT["fa:fa-code-branch Git Push"] --> LINT["fa:fa-code Lint & Format"]
  LINT --> TEST["fa:fa-flask Testes Unitarios"]
  TEST --> INTEG["fa:fa-flask Testes Integracao"]
  INTEG --> BUILD["fa:fa-box Build"]
```



```
BUILD --> SEC["fa:fa-lock Seguranca"]
```

```
SEC -->|"Falha"| FIX["fa:fa-bug Corrigir"]
```

```
SEC -->|"Passou"| DEPLOY["fa:fa-rocket Deploy"]
```

```
DEPLOY --> HEALTH["fa:fa-heart-pulse Health Check"]
```

```
HEALTH -->|"OK"| DONE["fa:fa-check Concluido"]
```

```
HEALTH -->|"Falha"| ROLLBACK["fa:fa-rotate-left Rollback"]
```

FIX --> GIT

ROLLBACK --> DEPLOY

`style GIT fill:#E3F2FD,stroke:#1A237E`

```
style LINT fill:#E3F2FD,stroke:#1A237E
```

```
style TEST fill:#E3F2FD,stroke:#1A237E
```

```
style INTEG fill:#E3F2FD,stroke:#1A237E
```

```
style BUILD fill:#F3E5F5,stroke:#7B1FA2
```

`style SEC fill:#FFF8E1,stroke:#F57F17`

```
style DEPLOY fill:#E8F5E9,stroke:#2E7D32
```

`style HEALTH fill:#E8F5E9,stroke:#2E7D32`

`style DONE fill:#E8F5E9,stroke:#2E7D32`

style FIX fill:#FFEBEE,stroke:#C62828

```
style ROLLBACK fill:#FFEBEE,stroke:#C62828
```

Fluxo de CI/CD completo. Cada estagio so avanca se o anterior passar. Seguranca e health check tem gates de aprovacao. Rollback automatico em caso de falha.

Continuous Integration (CI)

Todo push para branches compartilhadas dispara:

- Checkout do código
- Instalação de dependências
- Linting
- Testes unitários
- Testes de integração
- Build
- Análise de segurança

Continuous Delivery / Continuous Deployment (CD)

- ****Continuous Delivery:**** O artefato é gerado e publicado em um repositório, mas o deploy em produção é manual (aprovado por um humano).
- ****Continuous Deployment:**** O deploy em produção é automático após a pipeline CI passar.

GitHub Actions -- Exemplo Real

```
[yaml]
# .github/workflows/ci.yml
name: CI Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

env:
```

NODE_VERSION: 20.x

PNPM_VERSION: 9

jobs:


```
lint:
```

name: Lint e Formatação

```
runs-on: ubuntu-latest
```

steps:

- uses: actions/checkout@v4

- uses: pnpm/action-setup@v3

with:

```
version: ${ env.PNPM_VERSION }
```


- uses: actions/setup-node@v4

with:

```
node-version: ${ env.NODE_VERSION }
```

cache: pnpm

```
- run: pnpm install --frozen-lockfile
```

- run: pnpm lint

- run: pnpm format:check

test:

name: Testes

```
needs: lint
```

```
runs-on: ubuntu-latest
```

services:

postgres:

```
image: postgres:16-alpine
```

env:

POSTGRES_DB: app_test

```
POSTGRES_USER: app
```

POSTGRES_PASSWORD: secret

ports:

- 5432:5432

options: >-

```
--health-cmd pg_isready
```

```
--health-interval 10s
```



```
--health-timeout 5s
```

```
--health-retries 5
```

steps:

- uses: actions/checkout@v4

- uses: pnpm/action-setup@v3

with:

```
version: ${ env.PNPM_VERSION }
```

```
- uses: actions/setup-node@v4
```


with:

```
node-version: ${ env.NODE_VERSION }
```

cache: pnpm

```
- run: pnpm install --frozen-lockfile
```

```
- run: pnpm test:unit
```

```
- run: pnpm test:integration
```

env:

```
DATABASE_URL: postgres://app:secret@localhost:5432/app_test
```



```
build:
```

name: Build

```
needs: test
```

```
runs-on: ubuntu-latest
```

steps:

- uses: actions/checkout@v4

- uses: pnpm/action-setup@v3

with:

```
version: ${ env.PNPM_VERSION }
```

```
- uses: actions/setup-node@v4
```

with:

```
node-version: ${ env.NODE_VERSION }
```

cache: pnpm

```
- run: pnpm install --frozen-lockfile
```

- run: pnpm build

- uses: actions/upload-artifact@v4

with:

```
name: build-output
```

```
path: dist/
```

GitLab CI -- Exemplo Real

```
[yaml]
# .gitlab-ci.yml
stages:
  - lint
  - test
  - build
  - deploy

variables:
  NODE_VERSION: "20"
  PNPM_VERSION: "9"

cache:
  key: ${CI_COMMIT_REF_SLUG}
  paths:
    - node_modules/
    - .pnpm-store/

lint:
  stage: lint
  image: node:${NODE_VERSION}
  script:
    - npm install -g pnpm@${PNPM_VERSION}
    - pnpm install --frozen-lockfile
    - pnpm lint
    - pnpm format:check

test:
  stage: test
  image: node:${NODE_VERSION}
  services:
    - postgres:16-alpine
  variables:
    DATABASE_URL: postgres://app:secret@postgres:5432/app_test
    POSTGRES_DB: app_test
    POSTGRES_USER: app
    POSTGRES_PASSWORD: secret
  script:
```

```
- npm install -g pnpm@${PNPM_VERSION}
```

```
- pnpm install --frozen-lockfile
```

```
- pnpm test:unit
```



```
- pnpm test:integration
```



```
build:
```

```
stage: build
```

```
image: node:${NODE_VERSION}
```

script:

```
- npm install -g pnpm@${PNPM_VERSION}
```

```
- pnpm install --frozen-lockfile
```


- `pnpm build`

artifacts:

paths:

- dist/

```
expire_in: 30 days
```


deploy:

stage: deploy


```
image: alpine:latest
```

script:

```
- apk add --no-cache curl
```

```
- curl -X POST "${DEPLOY_HOOK_URL}"
```

only:

- main

```
when: manual
```

```
environment: production
```


3. Automação de Testes

Se tem uma coisa que aprendi na marra: teste que não roda no CI não existe. Pode ter 100% de cobertura na sua máquina -- se não passa na pipeline, é zero.
A pipeline de CI deve executar testes em camadas, respeitando a **pirâmide de testes**.

[Diagrama] Mermaid

```
graph TB
  subgraph PI[" "]
    direction TB
    E2E["E2E Tests<br/>Poucos | Lentos | Alto custo"]
    INTG["Integration Tests<br/>Alguns | Medios | Custo medio"]
    UNIT["Unit Tests<br/>Muitos | Rapidos | Baixo custo"]
  end
  E2E --> INTG
  INTG --> UNIT
  style E2E fill:#E3F2FD,stroke:#1A237E,color:#1A237E
  style INTG fill:#E8F5E9,stroke:#2E7D32
  style UNIT fill:#FFF8E1,stroke:#F57F17
```

Pirâmide de testes: muitos testes unitarios na base, poucos testes E2E no topo. Respeitar essa proporcao maximiza a velocidade e confianca da suite de testes.

Testes Unitários

[typescript]

```
// Exemplo com vitest
import { describe, it, expect } from 'vitest'
import { calculateDiscount } from './pricing'

describe('calculateDiscount', () => {
  it('aplica 10% para compras acima de R$ 100', () => {
    expect(calculateDiscount(150)).toBe(135)
  })

  it('não aplica desconto para compras abaixo de R$ 100', () => {
    expect(calculateDiscount(50)).toBe(50)
  })
})
```

})


```
it('lança erro para valores negativos', () => {
```

```
expect(() => calculateDiscount(-10)).toThrow('Valor inválido')
```

})

})

Testes de Integração

Testam a interação entre módulos, geralmente com banco de dados

```
[typescript]
import { describe, it, expect, beforeAll, afterAll } from 'vitest'
import { createApp } from './app'
import { prisma } from './lib/prisma'

const app = createApp()

describe('POST /users', () => {
  beforeAll(async () => {
    await prisma.$executeRawUnsafe('TRUNCATE TABLE users CASCADE')
  })

  afterAll(async () => {
    await prisma.$disconnect()
  })

  it('cria um usuário com dados válidos', async () => {
    const response = await app.inject({
      method: 'POST',
      url: '/users',
      payload: { name: 'João', email: 'joao@email.com' },
    })

    expect(response.statusCode).toBe(201)
    expect(response.json()).toHaveProperty('id')
  })
})
```

Testes E2E

```
[yaml]
# job no CI para E2E
e2e:
  name: Testes End-to-End
  needs: build
```


runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- uses: pnpm/action-setup@v3

with:

```
version: ${ env.PNPM_VERSION }
```

- uses: actions/setup-node@v4

with:


```
node-version: ${ env.NODE_VERSION }
```

cache: pnpm

```
- run: pnpm install --frozen-lockfile
```

- run: `pnpm build`

```
- run: pnpm dlx playwright install --with-deps
```

- run: `pnpm test:e2e`

env:

BASE_URL: `http://localhost:3000`

[typescript]

// E2E com Playwright

```
import { test, expect } from '@playwright/test'
```



```
test('usuário consegue finalizar compra', async ({ page }) => {
```

```
await page.goto('/produtos')
```

```
await page.click('text=Adicionar ao carrinho')
```

```
await page.click('text=Finalizar compra')
```



```
await page.fill('[name=email]', 'joao@email.com')
```

```
await page.click('text=Confirmar')
```



```
await expect(page.locator('text=Pedido confirmado')).toBeVisible()
```

})

4. Infraestrutura como Código (IaC)

Já entrei em servidor que "ninguém sabe como foi configurado". IaC existe pra acabar com essa frase de uma vez. Se sua infraestrutura não

Definicao: Infrastructure as Code (IaC) e o gerenciamento de infraestrutura (servidores, bancos, redes) atraves de arquivos de configuracao versionados. Sua infra vira um repositorio Git como qualquer outro.

Terraform (HashiCorp)

```
[hel]
# main.tf
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

backend "s3" {
  bucket = "meu-terraform-state"
  key    = "prod/terraform.tfstate"
  region = "us-east-1"
}

provider "aws" {
  region = "us-east-1"
}

resource "aws_ecs_service" "app" {
  name           = "app-service"
  cluster        = aws_ecs_cluster.main.id
  task_definition = aws_ecs_task_definition.app.arn
  desired_count  = 2
  launch_type    = "FARGATE"

  network_configuration {
```

```
subnets = var.private_subnet_ids
```

```
security_groups = [aws_security_group.app.id]
```


}


```
deployment_controller {
```

```
type = "CODE_DEPLOY" # blue/green
```

}

}

Pulumi

IaC com linguagens de programação reais (TypeScript, Python, Go,

```
[typescript]
import * as aws from '@pulumi/aws'
import * as pulumi from '@pulumi/pulumi'

const config = new pulumi.Config()
const stack = pulumi.getStack()

const cluster = new aws.ecs.Cluster('app-cluster')

const taskDefinition = new aws.ecs.TaskDefinition('app-task', {
  family: 'app',
  cpu: '256',
  memory: '512',
  networkMode: 'awsvpc',
  executionRoleArn: config.require('executionRoleArn'),
  containerDefinitions: JSON.stringify([
    {
      name: 'app',
      image: `meuregistro/app:${stack}`,
      portMappings: [{ containerPort: 3000 }],
    },
  ]),
})

new aws.ecs.Service('app-service', {
  cluster: cluster.arn,
  taskDefinition: taskDefinition.arn,
  desiredCount: 2,
  launchType: 'FARGATE',
  networkConfiguration: {
    subnets: config.requireObject<string[]>('privateSubnets'),
    securityGroups: [config.require('securityGroupId')],
  },
})
```

CloudFormation (AWS)

```
[yaml]
# template.yaml
AWSTemplateFormatVersion: "2010-09-09"
Description: "Stack da aplicação"

Parameters:
  Env:
    Type: String
    Default: production

Resources:
  ECSCluster:
    Type: AWS::ECS::Cluster
    Properties:
      ClusterName: !Sub "app-cluster-${Env}"

  AppService:
    Type: AWS::ECS::Service
    Properties:
      ServiceName: !Sub "app-service-${Env}"
      Cluster: !Ref ECSCluster
      LaunchType: FARGATE
      DesiredCount: 2
      TaskDefinition: !Ref AppTaskDefinition
      NetworkConfiguration:
        AwsvpcConfiguration:
          Subnets:
            - !Ref PrivateSubnet1
            - !Ref PrivateSubnet2
```

Pipeline para IaC

```
[yaml]
iac-plan:
  name: Terraform Plan
  runs-on: ubuntu-latest
  steps:
```


- uses: actions/checkout@v4

- uses: hashicorp/setup-terraform@v3

with:

```
terraform_version: 1.8.0
```

```
- run: terraform init
```

```
- run: terraform fmt -check
```

- run: terraform validate

```
- run: terraform plan -out=tfplan
```


- uses: actions/upload-artifact@v4

with:

```
name: tfplan
```

```
path: tfplan
```


iac-apply:

name: Terraform Apply

needs: iac-plan

runs-on: ubuntu-latest

```
if: github.ref == 'refs/heads/main'
```

```
environment: production
```

steps:

- uses: actions/checkout@v4

- uses: hashicorp/setup-terraform@v3

```
- run: terraform init
```

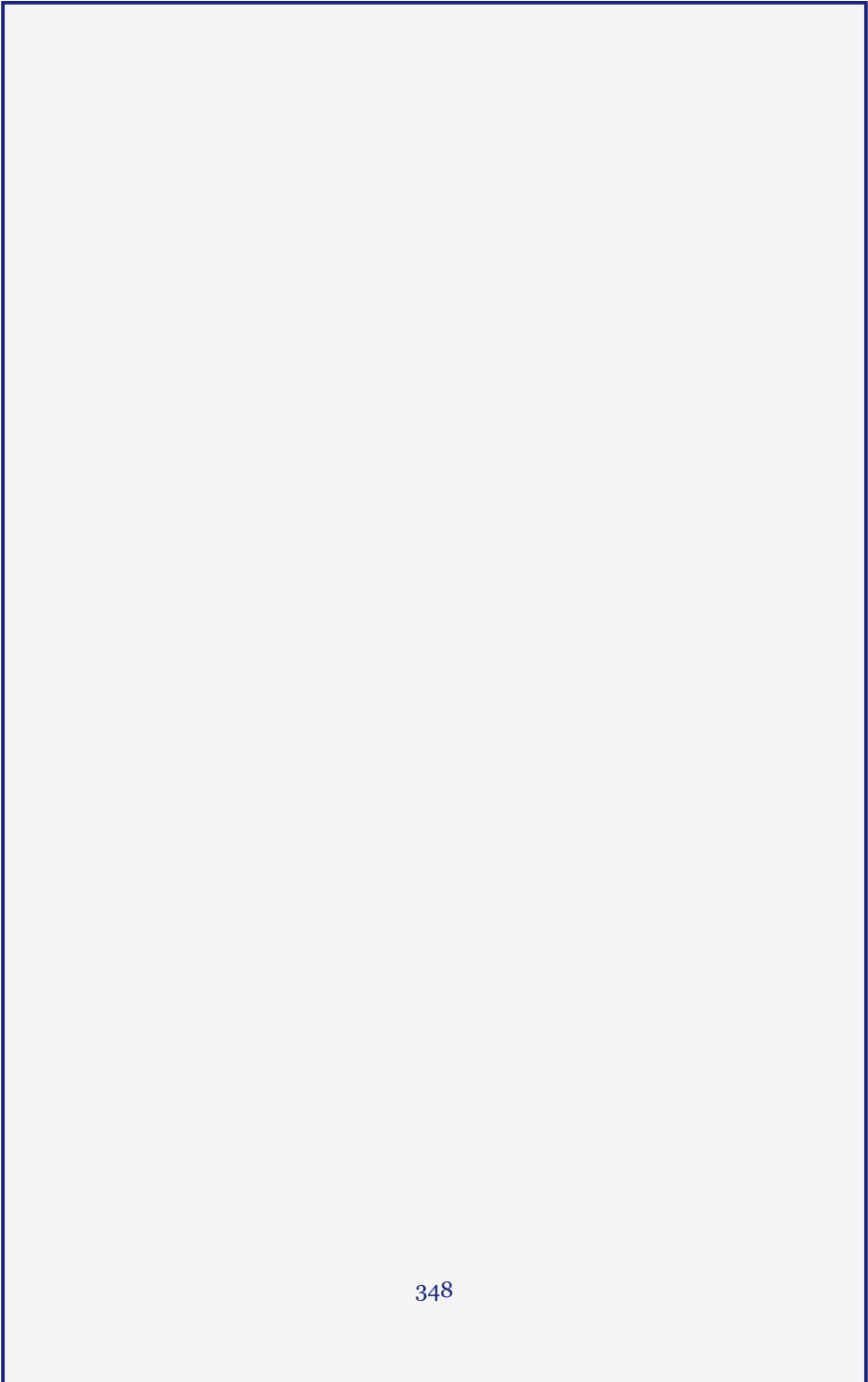
- uses: actions/download-artifact@v4

with:

```
name: tfplan
```

- run: terraform apply tfplan

Recapitulando: Automação cobre: CI/CD com pipelines, testes em 3 camadas (unitário, integração, E2E) e infraestrutura como código com Terraform, Pulumi ou



Recapitulando: Automação cobre: CI/CD com pipelines, testes em 3 camadas (unitário, integração, E2E) e infraestrutura como código com Terraform, Pulumi ou CloudFormation. Tudo versionado, tudo automatizado.

Deploy, Segurança e Operações

5. Automação de Deploys

Deploy sem estratégia de rollback não é deploy -- é aposta. Já vi um canary de 10% salvar o sábado inteiro de um time que soltou uma

Definicao: Estrategia de deploy e o metodo usado para colocar uma nova versao do software em producao sem (ou com minima) interrupcao para o usuario.

Blue-Green Deployment

Duas versões do ambiente (blue = atual, green = nova). O balanceador

[Diagrama] Mermaid

```
graph LR
    Users["Usuarios"] --> LB["Load Balancer"]
    LB -->|"Atual"| Blue["Blue (v1.0)"]
    LB -->|"Switch DNS"| Green["Green (v1.1)"]
    style Users fill:#E3F2FD,stroke:#1A237E
    style LB fill:#F3E5F5,stroke:#1A237E
    style Blue fill:#E3F2FD,stroke:#1565C0
    style Green fill:#E8F5E9,stroke:#2E7D32
```

Switch DNS/ALB aponta para Green. Rollback: reverter DNS para Blue.

Canary Deployment

Uma porcentagem pequena do tráfego vai para a nova versão. Aos

[yaml]

```
# Argo Rollouts para Kubernetes
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: app-rollout
spec:
  replicas: 10
```

```
revisionHistoryLimit: 2
```

selector:

`matchLabels:`

```
app: meu-app
```

template:

metadata:

labels:

```
app: meu-app
```

spec:

containers:

- name: app

```
image: meuregistro/app:1.1.0
```

strategy:

canary:

steps:

```
- setWeight: 10
```

```
- pause: { duration: 5m }
```

- `setWeight: 50`


```
- pause: { duration: 5m }
```

```
- setWeight: 100
```

[Diagrama] Mermaid

graph LR

```
LB2["Load Balancer"] -->|"10%"| V1_1["v1.1 (canary)"]
```

```
LB2 --> | "90%" | V1_0["v1.0 (stable)"]
```

```
V1_1 -->|"5 min"| V50["50%"]
```

```
V50 -->|"5 min"| V100["100%"]
```



```
style V1_1 fill:#FFF8E1,stroke:#F57F17
```

```
style V1_0 fill:#E3F2FD,stroke:#1565C0
```

```
style V50 fill:#FFF8E1,stroke:#F57F17
```

`style V100 fill:#E8F5E9,stroke:#2E7D32`

Atencao: Canary exige monitoramento em tempo real. Se a taxa de erro subir no

Atenção: Canary exige monitoramento em tempo real. Se a taxa de erro subir nos primeiros 10%, o canary deve abortar automaticamente.

Rolling Update

```
[yaml]
# Docker Swarm com rolling update
version: "3.8"
services:
  app:
    image: meuregistro/app:${VERSION}
    deploy:
      replicas: 3
      update_config:
        parallelism: 1
        delay: 10s
        order: start-first
      rollback_config:
        parallelism: 1
        order: stop-first
```

Feature Flags

Permittem ativar/desativar funcionalidades em produção sem fazer

```
[typescript]
import { createClient } from '@configu/sdk'

const configu = createClient({ apiKey: process.env.CONFIGU_KEY })

async function handler(req, res) {
  const novoCheckout = await configu.getFlag('checkout-v2-enabled')

  if (novoCheckout && req.user.id in novoCheckout.targeting) {
    return handleCheckoutV2(req, res)
  }

  return handleCheckoutV1(req, res)
}
```

Dica: Ferramentas: LaunchDarkly, Configu, GrowthBook, Unleash, Flagsmith.



Dica: Ferramentas: LaunchDarkly, Configu, GrowthBook, Unleash, Flagsmith.

Comece com uma gratuita (Unleash) e migre quando precisar de targeting complexo.

6. Automação de Banco de Dados

Banco de dados é a parte que mais me tira o sono quando o assunto é automação. Migração automática é ótima -- desde que você tenha

Atenção: Migração automática em produção sem backup prévio e a causa mais comum de indisponibilidade. Sempre faça backup antes.

Migrations Automáticas no CI

```
[yaml]
migrate:
  name: Rodar Migrations
  needs: build
  runs-on: ubuntu-latest
  environment: ${ github.ref == 'refs/heads/main' && 'production' || 'staging' }
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
    - uses: actions/setup-node@v4
    - run: pnpm install --frozen-lockfile
    - run: pnpm build
    - uses: actions/download-artifact@v4
      with:
        name: build-output
        path: dist
    - run: pnpm db:migrate
    env:
      DATABASE_URL: ${ secrets.DATABASE_URL }
```

Exemplo com Prisma Migrate

```
[typescript]
// scripts/migrate.ts
import { execSync } from 'node:child_process'

const env = process.env.NODE_ENV || 'development'
```



```
function run() {
```

```
console.log('[EXECUTANDO] Rodando migrations em ' + env + '...')
```



```
execSync('pnpm prisma migrate deploy', {
```

```
stdio: 'inherit',
```

```
env: { ...process.env, DATABASE_URL: process.env.DATABASE_URL },
```

})


```
console.log('[BOM] Migrations executadas com sucesso')
```

}


```
run()
```

Seed Automático em CI

```
[typescript]
// scripts/seed.ts
import { PrismaClient } from '@prisma/client'

const prisma = new PrismaClient()

async function seed() {
  console.log('[SEED] Iniciando seed...')

  await prisma.tenant.create({
    data: {
      name: 'Default',
      slug: 'default',
      users: {
        create: {
          email: 'admin@empresa.com',
          role: 'ADMIN',
        },
      },
    },
  })

  console.log('[BOM] Seed concluído')
}

seed()
  .catch((e) => {
    console.error(e)
    process.exit(1)
  })
  .finally(() => prisma.$disconnect())
```

Rollback Automático

[Diagrama] Mermaid

graph TD

BACKUP["pg_dump antes da migration"] --> MIGRATE["Rodar migration"]

```
MIGRATE -->|"Sucesso" | DONE_M["Concluido"]
```

```
MIGRATE -->|"Falha"| RESTORE["Restaurar backup"]
```



```
RESTORE --> NOTIFY_SLACK["Notificar time"]
```

`style BACKUP fill:#E3F2FD,stroke:#1A237E`

style MIGRATE fill:#FFF8E1,stroke:#F57F17

```
style DONE_M fill:#E8F5E9,stroke:#2E7D32
```

```
style RESTORE fill:#FFEBEE,stroke:#C62828
```

```
style NOTIFY_SLACK fill:#FFEBEE,stroke:#C62828
```

Estratégia: antes de rodar a migration, fazer backup; se a migration

```
[yaml]
- name: Backup antes da migration
  run: pg_dump $DATABASE_URL > /tmp/pre_migration_backup.sql
  env:
    DATABASE_URL: ${ secrets.DATABASE_URL }

- name: Rodar migration
  id: migrate
  run: pnpm db:migrate
  continue-on-error: true
  env:
    DATABASE_URL: ${ secrets.DATABASE_URL }

- name: Rollback em caso de falha
  if: steps.migrate.outcome != 'success'
  run: |
    echo "[ROLLBACK] Migration falhou -- restaurando backup..."
    psql $DATABASE_URL < /tmp/pre_migration_backup.sql
    echo "[BOM] Backup restaurado"
  env:
    DATABASE_URL: ${ secrets.DATABASE_URL }
```

O `continue-on-error: true` é essencial aqui -- sem ele, o pipeline aborta antes de chegar no passo de rollback.

7. Automação de Segurança

Segurança automatizada não substitui um bom engenheiro, mas pega 80% dos problemas antes do code review. E o melhor: não

[Diagrama] Mermaid

```
graph LR
  CODE["Codigo"] --> SAST["SAST<br/>Analise Estatica"]
  SAST -->|"Passou"| BUILD_CONT["Build"]
  BUILD_CONT --> TRIVY["Trivy<br/>Container Scan"]
  TRIVY -->|"Passou"| DEPLOY_CONT["Deploy"]
  DEPLOY_CONT --> DAST["DAST<br/>Teste Dinamico"]
  SAST -->|"Falha"| FIX_CONT["Corrigir"]
```

DAST -->|"Falha"| FIX_CONT

TRIVY -->|"Falha"| FIX_CONT

`FIX_CONT --> CODE`

```
style SAST fill:#E3F2FD,stroke:#1A237E
```

```
style TRIVY fill:#FFF8E1,stroke:#F57F17
```

`style DAST fill:#F3E5F5,stroke:#7B1FA2`

```
style FIX_CONT fill:#FFEBEE,stroke:#C62828
```

Pipeline de segurança em 3 camadas: estática no commit, container no build, dinâmica no deploy.

SAST (Static Application Security Testing)

```
[yaml]
security-sast:
  name: Análise Estática (SAST)
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
    - uses: actions/setup-node@v4
    - run: pnpm install --frozen-lockfile

    # ESLint com plugins de segurança
    - name: ESLint Security
      run: pnpm dlx eslint . --ext .ts --rulesdir eslint-security-rules

    # SonarQube
    - name: SonarQube Scan
      uses: sonarsource/sonarcloud-github-action@master
      env:
        SONAR_TOKEN: ${ secrets.SONAR_TOKEN }
      with:
        args: >
          -Dsonar.organization=minha-org
          -Dsonar.projectKey=meu-projeto
          -Dsonar.javascript.lcov.reportPaths=coverage/lcov.info

    # Trivy para vulnerabilidades em container
    - name: Trivy Scan
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: meuregistro/app:${ secrets.github.sha }
        format: sarif
        output: trivy-results.sarif
```

DAST (Dynamic Application Security Testing)

```
[yaml]
security-dast:
  name: Teste Dinâmico (DAST)
  needs: deploy-staging
  runs-on: ubuntu-latest
  steps:
    - name: OWASP ZAP Scan
      uses: zapproxy/action-full-scan@v0.10
      with:
        target: https://staging.meusistema.com.br
        rules_file_name: .zap/rules.tsv
        cmd_options: "-a -j"
```

Dependency Scanning

```
[yaml]
dependency-scan:
  name: Varredura de Dependencias
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
    - uses: actions/setup-node@v4
    - run: pnpm install --frozen-lockfile

  # Snyk
  - name: Snyk Scan
    uses: snyk/actions/node@master
    env:
      SNYK_TOKEN: ${ secrets.SNYK_TOKEN }
    with:
      args: --severity-threshold=medium

  # npm audit
  - name: npm audit
    run: pnpm audit --audit-level=high
```

Dica: Configure Dependabot ou Renovate para abrir PRs automaticamente quando uma dependencia tem CVE conhecida. Isso reduz o tempo de exposicao de dias para



Dica: Configure Dependabot ou Renovate para abrir PRs automaticamente quando uma dependencia tem CVE conhecida. Isso reduz o tempo de exposicao de dias para

horas.

8. Automação de Code Review

Code review automatizado é aquele colega chato que sempre aponta o óbvio -- e é por isso que você precisa dele. Bots não substituem revisão humana, mas eliminam o ruído.

Bots no Pipeline

[yaml]

```
code-review:
  name: Revisao Automatica
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0

    # Linters automáticos
    - uses: pnpm/action-setup@v3
    - uses: actions/setup-node@v4
    - run: pnpm install --frozen-lockfile
    - run: pnpm lint
    - run: pnpm format:check

    # Status checks
    - name: Verificar tamanho do PR
      uses: actions/github-script@v7
      with:
        script: |
          const { data: pr } = await github.rest.pulls.get({
            ...context.repo,
            pull_number: context.issue.number,
          })
          const changedFiles = pr.changed_files
          const additions = pr.additions

          if (changedFiles > 20) {
            core.warning('[WARNING] PR grande: ' + changedFiles + ' arquivos a
```

}

```
if (additions > 500) {
```

```
core.warning('[WARNING] Muitas linhas adicionadas: ' + additions)
```

}

CodeQL


```
- name: Initialize CodeQL
```

```
uses: github/codeql-action/init@v3
```

with:

```
languages: javascript-typescript
```


- name: CodeQL Analysis

uses: github/codeql-action/analyze@v3

with:

category: /language:javascript

PRs com mais de 20 arquivos ou 500 linhas tem correlacao direta com maior taxa de

PRs com mais de 20 arquivos ou 500 linhas tem correlacao direta com maior taxa de defeitos. O bot nao bloqueia, mas alerta o revisor para redobrar a atencao.

Dependabot Configuration

```
[yaml]
# .github/dependabot.yml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "weekly"
      day: "monday"
    open-pull-requests-limit: 10
    labels:
      - "dependencies"
      - "automated"
    reviewers:
      - "time-squad"
    commit-message:
      prefix: "chore"
      include: "scope"
```

9. Automação de Releases

Já perdi a conta de quantas vezes um release manual quebrou algo porque alguém esqueceu de atualizar a versão. O semantic-release foi um dos melhores investimentos de automação que já fiz num

Definicao: Semantic Versioning (SemVer): versionamento no formato MAJOR.MINOR.PATCH. MAJOR = quebra compatibilidade, MINOR = nova feature, PATCH = correcao de bug.

Semantic Versioning Automático

```
[yaml]
release:
  name: Gerar Release
```

```
needs: build
```

```
runs-on: ubuntu-latest
```

```
if: github.ref == 'refs/heads/main'
```

permissions:

contents: write


```
packages: write
```

steps:

- uses: actions/checkout@v4

with:

fetch-depth: 0

Analise de commits para versionamento semantico

- name: Semantic Release

uses: cycjimmy/semantic-release-action@v4

id: semantic

env:

```
GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
```

```
NPM_TOKEN: ${ secrets.NPM_TOKEN }
```

with:

```
extra_plugins: |
```

@semantic-release/changelog

@semantic-release/git

@semantic-release/npm

- name: Publicar no npm

```
if: steps.semantic.outputs.new_release_published == 'true'
```

run: |

```
echo '[PUBLISH] Publicando versao ${ steps.semantic.outputs.new_relea
```

```
npm publish
```


env:

```
NODE_AUTH_TOKEN: ${ secrets.NPM_TOKEN }
```

[Diagrama] Mermaid

graph LR

```
COMMITTS["Conventional Commits<br/>feat: / fix: / chore:"] --> SEMANTIC["semantic-rel
```

```
SEMANTIC --> VERSION["Calcula versao<br/>1.4.0 -> 1.5.0"]
```

```
VERSION --> CHANGELOG["Gera CHANGELOG.md"]
```

```
CHANGELOG --> GIT_TAG["Cria Git Tag<br/>v1.5.0"]
```



```
GIT_TAG --> PUBLISH["Publica pacote"]
```

```
PUBLISH --> RELEASE["Cria GitHub Release"]
```

style COMMITS fill:#E3F2FD,stroke:#1A237E

```
style SEMANTIC fill:#F3E5F5,stroke:#7B1FA2
```

```
style GIT_TAG fill:#FFF8E1,stroke:#F57F17
```

`style RELEASE fill:#E8F5E9,stroke:#2E7D32`

Conventional Commits

```
feat: adiciona calculadora de impostos
feat(api): nova rota de relatorios
fix: corrige validacao de CPF
fix(auth): timeout na renovacao do token
chore: atualiza dependencias
docs: atualiza README
refactor: extrai logica de pagamento para servico
perf: otimiza consulta de historico
test: adiciona testes para o modulo de notas fiscais
```

Changelog Gerado Automaticamente

```
[markdown]
# Changelog

## [1.5.0](https://github.com/org/repo/compare/v1.4.0...v1.5.0) (2025-06-15)

### Features
* **pagamento:** adiciona suporte a PIX ([abc1234](https://github.com/org/repo/compare/v1.4.0...v1.5.0))
* **relatorios:** nova rota de exportacao CSV ([def5678](https://github.com/org/repo/compare/v1.4.0...v1.5.0))

### Bug Fixes
* **auth:** corrige timeout na renovacao do token ([ghi9012](https://github.com/org/repo/compare/v1.4.0...v1.5.0))
* **validacao:** CPF com formatacao agora e aceito ([jkl3456](https://github.com/org/repo/compare/v1.4.0...v1.5.0))
```

Dica: O changelog gerado automaticamente e o melhor documentacao viva que existe.

Ele reflete exatamente o que foi entregue, sem depender de memoria humana.

10. Automação de Ambientes

Ambiente efêmero é um daqueles luxos que vira necessidade depois que você acostuma. Testar em produção deveria ser exceção, não

Definicao: Ambiente efemero (ephemeral environment): um ambiente completo (servico + banco) criado automaticamente para uma branch/PR e destruido quando a branch e mesclada ou fechada.

Ephemeral Environments

Ambientes temporários criados automaticamente para cada branch de

```
if [yaml]
deploy-preview:
  name: Deploy Preview
  runs-on: ubuntu-latest
  if: github.event_name == 'pull_request'
  environment:
    name: preview-${{ github.event.number }}
    url: https://pr-${{ github.event.number }}.meusistema.com.br
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
    - uses: actions/setup-node@v4
    - run: pnpm install --frozen-lockfile
    - run: pnpm build

    # Criar banco efêmero
    - name: Criar database preview
      run: |
        aws rds create-db-instance \
          --db-instance-identifier app-preview-${{ github.event.number }} \
          --db-instance-class db.t3.micro \
          --engine postgres \
          --master-username preview \
          --master-user-password ${ secrets.PREVIEW_DB_PASSWORD }

    - name: Rodar migrations
      run: pnpm db:migrate
      env:
        DATABASE_URL: postgres://preview:${ secrets.PREVIEW_DB_PASSWORD }@pr

    - name: Fazer deploy no ECS
      run: |
        aws ecs update-service \
          --cluster preview \
          --service app-preview-${{ github.event.number }} \
          --force-new-deployment
```


destroy-preview:

name: Destruir Preview

```
runs-on: ubuntu-latest
```

```
if: github.event_name == 'pull_request' && github.event.action == 'closed'
```

steps:

- name: Remover database preview

```
run: |
```



```
aws rds delete-db-instance \
```

```
--db-instance-identifier app-preview-${{ github.event.number }} \
```

`--skip-final-snapshot`

- name: Remover serviço ECS

run: |

```
aws ecs delete-service \
```

```
--cluster preview \
```



```
--service app-preview-${{ github.event.number }} \
```

--force

Atencao: Nao esqueca de destruir ambientes efemeros quando a branch e fechada.

Atenção: Não esqueça de destruir ambientes efêmeros quando a branch é fechada. Ambiente esquecido custa dinheiro. Use um cron job de limpeza como fallback.

Preview Deployments (Vercel / Render)

```
[yaml]
vercel-preview:
  name: Vercel Preview
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: amondnet/vercel-action@v25
      with:
        vercel-token: ${ secrets.VERCEL_TOKEN }
        vercel-org-id: ${ secrets.VERCEL_ORG_ID }
        vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }
        github-token: ${ secrets.GITHUB_TOKEN }
        github-comment: true
```

11. Automação de Monitoramento

Monitoramento sem ação automática é só alarmismo. A diferença entre um sistema robusto e um frágil é o que acontece depois do alerta -- e quem (ou o que) age.

Alertas Automáticos

```
[yaml]
monitoring-setup:
  name: Configurar Alertas
  runs-on: ubuntu-latest
  steps:
    - name: Configurar Datadog Monitors
      uses: datadog/synthetics-ci-github-action@v1
      with:
        api_key: ${ secrets.DD_API_KEY }
        app_key: ${ secrets.DD_APP_KEY }
        public_ids: ${ vars.DATADOG_SYNTHETIC_TESTS }
```

- name: Configurar alerta de latência P99

run: |

```
curl -X POST "https://api.datadoghq.com/api/v1/monitor" \
```

```
-H "Content-Type: application/json" \
```



```
-H "DD-API-KEY: ${ secrets.DD_API_KEY }" \
```

```
-H "DD-APPLICATION-KEY: ${ secrets.DD_APP_KEY }" \
```

-d '{

```
"name": "Latencia P99 > 500ms",
```

```
"type": "query alert",
```

```
"query": "avg(last_5m):p99(trace.http.request.duration{service:app
```

```
"message": "Latencia alta detectada! @time-squad",
```

```
"tags": ["service:app", "severity:critical"]
```


}'

Self-Healing

[Diagrama] Mermaid

```
graph TD
    CHECK["Verificar runningCount<br/>vs desiredCount"] -->|"running < desired"| DEPLOY_FORCE["DEPLOY_FORCE"]
    CHECK -->|"running == desired"| HEALTHY["Service saudável"]
    DEPLOY_FORCE --> NOTIFY["Notificar Slack"]
    NOTIFY --> WAIT["Aguardar 60s"]
    WAIT --> CHECK
    style CHECK fill:#FFF8E1,stroke:#F57F17
    style DEPLOY_FORCE fill:#E3F2FD,stroke:#1A237E
    style HEALTHY fill:#E8F5E9,stroke:#2E7D32
    style NOTIFY fill:#F3E5F5,stroke:#7B1FA2
```

[typescript]

```
// scripts/self-healing.ts
import { ECSClient, DescribeServicesCommand, UpdateServiceCommand } from '@aws-sdk/client-ecs'

const ecs = new ECSClient({ region: 'us-east-1' })

async function healService(cluster: string, service: string) {
  const { services } = await ecs.send(
    new DescribeServicesCommand({ cluster, services: [service] })
  )

  const svc = services![0]
  const running = svc.runningCount ?? 0
  const desired = svc.desiredCount ?? 0

  if (running < desired) {
    console.log('[REPAIR] Service ' + service + ' com ' + running + '/' + desired)

    await ecs.send(
      new UpdateServiceCommand({
        cluster,
        service,
        forceNewDeployment: true,
      })
    )
  }
}
```

```
console.log('[BOM] Deploy forçado enviado')
```

```
} else {
```

```
console.log('[BOM] Service ' + service + ' saudavel (' + running + '/' + d
```

}

}


```
healService('production', 'app-service').catch(console.error)
```

Recapitulando: Self-healing não substitui um SRE, mas reduz o MTTR (Mean Time To Recovery) de minutos para segundos em cenários comuns (container crashando,

Recapitulando: Self-healing não substitui um SRE, mas reduz o MTTR (Mean Time To Recovery) de minutos para segundos em cenários comuns (container crashando, node sendo drenado).

Runbooks Automáticos

```
[yaml]
# runbook: recuperacao-banco.yml
name: Runbook - Recuperar Database
on:
  workflow_dispatch:
    inputs:
      database_name:
        description: 'Nome do banco'
        required: true
      restore_point:
        description: 'PITR timestamp (YYYY-MM-DD HH:MM:SS)'
        required: false

jobs:
  restore:
    runs-on: ubuntu-latest
    steps:
      - name: Validar parametros
        run: |
          if [ -z "${{ inputs.database_name }}" ]; then
            echo "[RUIM] database_name e obrigatorio"
            exit 1
          fi

      - name: Parar aplicacao
        run: |
          aws ecs update-service \
            --cluster production \
            --service app-service \
            --desired-count 0

      - name: Restaurar snapshot
        run: |
          if [ -n "${{ inputs.restore_point }}" ]; then
            aws rds restore-db-instance-to-point-in-time \
```

```
--source-db-instance-identifier ${ inputs.database_name } \
```

```
--target-db-instance-identifier ${ inputs.database_name }--rest
```

```
--restore-time "${ inputs.restore_point }"
```

`else`

```
SNAPSHOT=$(aws rds describe-db-snapshots \
```



```
--db-instance-identifier ${ inputs.database_name } \
```

```
--query "DBSnapshots[-1].DBSnapshotIdentifier" \
```

```
--output text)
```

```
aws rds restore-db-instance-from-db-snapshot \
```

```
--db-instance-identifier ${ inputs.database_name }-restored \
```

```
--db-snapshot-identifier $SNAPSHOT
```

fi

- name: Trocar DNS para DB restaurado

run: |

```
aws rds modify-db-instance \
```

```
--db-instance-identifier ${ inputs.database_name } \
```

```
--new-db-instance-identifier ${ inputs.database_name }-old
```

```
aws rds modify-db-instance \
```

```
--db-instance-identifier ${ inputs.database_name }-restored \
```

```
--new-db-instance-identifier ${ inputs.database_name }
```


- name: Subir aplicacao

run: |

```
aws ecs update-service \
```

```
--cluster production \
```

```
--service app-service \
```

```
--desired-count 3
```


- name: Notificar

run: |

```
curl -X POST ${ secrets.SLACK_WEBHOOK } \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"text": "[BOM] Database ${ inputs.database_name } recuperad
```

12. Pipeline as Code -- Boas Práticas

Depois de centenas de pipelines escritas e reescritas, minha maior lição é simples: pipeline é código, trate como tal. Review, versionamento, testes -- tudo que vale para seu app vale para sua pipeline.

Modularização e Reuso

```
[yaml]
# .github/actions/setup-node/action.yml
name: "Setup Node.js"
description: "Configura Node.js, pnpm e cache"
inputs:
  node-version:
    required: false
    default: "20.x"
  pnpm-version:
    required: false
    default: "9"

runs:
  using: "composite"
  steps:
    - uses: actions/checkout@v4
    - uses: pnpm/action-setup@v3
      with:
        version: "${{ inputs.pnpm-version }}"
    - uses: actions/setup-node@v4
      with:
        node-version: "${{ inputs.node-version }}"
        cache: pnpm
    - run: pnpm install --frozen-lockfile
      shell: bash
```

```
[yaml]
# .github/workflows/ci.yml
jobs:
  lint:
```

```
runs-on: ubuntu-latest
```

steps:


```
- uses: ./github/actions/setup-node
```

```
- run: pnpm lint
```



```
test:
```

```
runs-on: ubuntu-latest
```

steps:

```
- uses: ./github/actions/setup-node
```

- run: pnpm test

Reuso de Jobs com Matrix

```
[yaml]
test-matrix:
  name: Testes (${ matrix.node-version })
  runs-on: ubuntu-latest
  strategy:
    matrix:
      node-version: [18.x, 20.x, 22.x]
      fail-fast: false
  steps:
    - uses: ./github/actions/setup-node
      with:
        node-version: ${ matrix.node-version }
    - run: npm test:ci
```

Pipeline de Deploy com Environments e Gates

```
[yaml]
deploy-production:
  name: Deploy em Producao
  needs: [lint, test, build, security-sast]
  runs-on: ubuntu-latest
  if: github.ref == 'refs/heads/main'
  environment:
    name: production
    url: https://meusistema.com.br
  concurrency:
    group: production-deploy
    cancel-in-progress: false
  steps:
    - uses: actions/checkout@v4
    - uses: ./github/actions/setup-node
    - uses: actions/download-artifact@v4
      with:
        name: build-output
        path: dist

    - name: Login no Docker Registry
```

```
uses: docker/login-action@v3
```

with:

```
username: ${ secrets.DOCKER_USER }}
```

```
password: ${{ secrets.DOCKER_PASS }}
```


- name: Build e Push da Imagem

```
run: |
```



```
docker build -t meuregistro/app:${{ github.sha }} .
```

```
docker push meuregistro/app:${{ github.sha }}
```


- name: Deploy no ECS

```
run: |
```

```
aws ecs update-service \
```

```
--cluster production \
```

```
--service app-service \
```


--force-new-deployment

- name: Verificar Saude

```
run: |
```

```
for i in {1..30}; do
```

```
STATUS=$(curl -s -o /dev/null -w "%{http_code}" https://meusistema.c
```

```
if [ "$STATUS" = "200" ]; then
```

```
echo "[BOM] Deploy saudavel"
```



```
exit 0
```

fi

```
sleep 10
```

done

```
echo "[RUIM] Health check falhou apos deploy"
```

```
exit 1
```


- name: Rollback automatico


```
if: failure()
```

```
run: |
```

```
echo "[ROLLBACK] Iniciando rollback..."
```

```
aws ecs update-service \
```

```
--cluster production \
```

```
--service app-service \
```

--force-new-deployment

Checklist de Boas Práticas

Depois de anos ajustando pipelines, minha checklist pessoal é esta:

1. Pipeline rápido: Cada job deve rodar em menos de 10 minutos
2. **Fail fast:** jobs mais rápidos primeiro (lint antes de e2e)
3. **Cache inteligente:** dependências, build, Docker layers
4. **Secrets:** nunca hardcode -- use secrets do provedor
5. **Idempotência:** rodar o pipeline 2x deve dar o mesmo resultado
6. **Notificações:** notifique o time em caso de falha (Slack, email)
7. **Versionamento de pipeline:** a pipeline é código -- versionada, revisada, testada
8. **Ambiente imutável:** nunca faça alterações manuais em produção
9. **Observabilidade:** métricas de tempo de pipeline, taxa de falha, MTTR
10. **Documentação:** cada job deve ter `name` descritivo e comentários quando necessário

Recapitulando: Automação de deploy, banco, segurança, code review, releases, ambientes e monitoramento formam o ciclo completo de operações. Cada peça reduz

risco, aumenta velocidade e liberta o time para focar no que importa: entregar valor.

Exercício Prático

Pipeline de Deploy com Rollback Automático

Crie uma pipeline completa de deploy que:

- Execute lint e testes unitários
- Faça build da imagem Docker
- Execute varredura de segurança (Trivy)

- Faça deploy em um ambiente de staging
- Execute health check
- Em caso de falha, faça rollback automático
- Notifique o time no Slack

Requisitos:

1. Use GitHub Actions

- Inclua pelo menos uma action reutilizável
- Use `needs` para dependência entre jobs
- Configure environment protection rules no GitHub
- Simule o rollback com `if: failure()`

```
[yaml]
name: Deploy Pipeline

on:
  push:
    branches: [main]

jobs:
  # Seu código aqui
```

Critérios de correção:

1. Pipeline completa com todos os 7 passos (2 pts)
2. Action reutilizável corretamente definida e usada (2 pts)
3. Dependências entre jobs com `needs` (1 pt)
4. Environment protection configurada (2 pts)
5. Rollback com `if: failure()` (1 pt)
6. Notificação Slack (1 pt)
7. Pipeline documentada com `name` descritivos (1 pt)

Resumo do Capítulo

Conceito	Ferramentas	Objetivo
----------	-------------	----------

Deploy

Blue-green, Canary, Rolling

Zero downtime e rollback
seguro

Banco

Prisma Migrate, pg_dump

Schema versionado e
migrações seguras

Segurança

SonarQube, Snyk, Trivy, OWASP ZAP

Vulnerabilidades
detectadas no pipeline

Code Review

Dependabot, ESLint	CodeQL,
-----------------------	---------

Revisão automatizada de
código

Release

semantic-release

Versionamento e changelog
automáticos

Ambientes

Vercel, efêmero	Render,	ECS
--------------------	---------	-----

Preview de cada branch

Monitoramento

Datadog, self-healing, runbooks

Alertas e recuperação
automática

Pipeline as Code

GitHub Actions, GitLab CI

Infraestrutura de CI/CD
modular e reutilizável

Infraestrutura como Código e Pipeline Visual

1. Infraestrutura como Código (IaC)

Era uma segunda-feira à noite. Um colega precisava subir um ambiente de staging novo para o cliente testar uma feature. Ele abriu o console da AWS, começou a clicar em "Create instance", configurou security groups manualmente, anotou o IP num post-it. Três horas depois, o ambiente estava pronto -- mas não era idêntico ao de produção. Dois dias mais tarde, o bug que o cliente encontrou não se reproduzia em staging porque a configuração era diferente. Esse cenário se repete em milhares de empresas: ambientes configurados na mão, diferenças entre staging e produção, e a famosa frase "na minha máquina funciona".

Infrastructure as Code (IaC) resolve exatamente esse problema: você descreve sua infraestrutura em arquivos de configuração

Definicao: Infrastructure as Code (IaC) e a pratica de gerenciar e provisionar infraestrutura -- servidores, redes, bancos, balanceadores -- atraves de arquivos de definicao legiveis por maquinas, em vez de configuracao manual ou scripts

interativos. O mesmo arquivo que cria o ambiente de staging cria o de producao.

Por que IaC importa

Beneficio	Descrição
Reprodutibilidade	O mesmo código gera ambientes idênticos -- fim do "funciona na minha máquina"
Versionamento	Cada mudança na infra fica registrada no Git com autor, data e motivo
Auditoria	`git log` mostra exatamente quem alterou o que e quando

****Velocidade****

Ambiente novo em minutos, não em dias

****Consistência****

Staging, produção e disaster recovery
usam a mesma definição

****Rollback****

Reverteu o commit? Reverteu a infra.

Terraform vs Pulumi vs CloudFormation

Característica	Terraform	Pulumi	CloudFormation
Linguagem	HCL (HashiCorp Config Language)	TypeScript, Python, Go, C#, Java	JSON ou YAML
Provedores	Multi-cloud (AWS, Azure, GCP, +300)	Multi-cloud (AWS, Azure, GCP, Kubernetes)	Apenas AWS
Estado (State)	Arquivo local ou remoto (S3, Terraform Cloud)	Arquivo local ou remoto (S3, Pulumi Cloud)	Gerenciado automaticamente pela AWS
Modularidade	Modules registry	Packages npm/PyPI/Go	Nested stacks
Preview	`terraform plan`	`pulumi preview`	`aws cloudformation deploy`
Curva de aprendizado	Média (HCL é simples)	Média-alta (exige linguagem de programação)	Baixa (YAML/JSON, mas verboso)
Versionamento de estado	Bloqueio otimista nativo	Bloqueio otimista nativo	Gerenciado pela AWS
Open source	Sim	Sim	Não (proprietário AWS)
Melhor para	Equipes multi-cloud ou que querem portabilidade	Equipes que preferem TypeScript/Python para infra	Equipes 100% AWS que querem integração nativa

Exemplo: Terraform para uma instância EC2

```
[hel]
terraform {
  required_version = ">= 1.6"
  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
```

}

}

}


```
provider "aws" {
```

```
region = "us-east-1"
```

}


```
resource "aws_security_group" "web_sg" {
```

```
name = "web-server-sg"
```

```
description = "Permite HTTP e SSH"
```



```
ingress {
```

```
from_port = 80
```

```
to_port      = 80
```



```
protocol    = "tcp"
```

```
cidr_blocks = ["0.0.0.0/0"]
```

}


```
ingress {
```

```
from_port = 22
```

```
to_port      = 22
```

```
protocol    = "tcp"
```



```
cidr_blocks = var.ssh_allowed_cidrs
```

}


```
egress {
```

```
from_port = 0
```

```
to_port      = 0
```

```
protocol = "-1"
```

```
cidr_blocks = ["0.0.0.0/0"]
```


}


```
tags = {
```

```
Name      = "web-server-sg"
```

```
Managed = "terraform"
```

}

}


```
resource "aws_instance" "web" {
```

```
ami = data.aws_ami.ubuntu.id
```

```
instance_type = var.instance_type
```

```
vpc_security_group_ids = [aws_security_group.web_sg.id]
```



```
user_data = <<-EOF
```

```
#!/bin/bash
```

```
apt-get update
```



```
apt-get install -y nginx
```

```
systemctl enable nginx
```

```
systemctl start nginx
```

EOF


```
tags = {
```

```
Name      = "web-server-${var.environment}"
```

```
Managed = "terraform"
```


}

}


```
data "aws_ami" "ubuntu" {
```

```
most_recent = true
```

```
owners      = ["099720109477"]
```



```
filter {
```



```
name    = "name"
```

```
values = ["ubuntu/images/hvm-ssd/ubuntu-22.04-amd64-server-*"]
```

}

}


```
variable "instance_type" {
```

```
description = "Tipo da instancia EC2"
```

```
type      = string
```



```
default      = "t3.micro"
```

}


```
variable "environment" {
```

```
description = "Ambiente (dev, staging, production)"
```

```
type      = string
```

}


```
variable "ssh_allowed_cidrs" {
```

```
description = "CIDRs permitidos para SSH"
```

```
type          = list(string)
```

```
default      = ["10.0.0.0/8"]
```

}


```
output "web_public_ip" {
```

```
description = "IP publico do servidor web"
```



```
value = aws_instance.web.public_ip
```

}

Gerenciamento de Estado

O *estado* (state) é o coração do Terraform. Ele mapeia os recursos do

Atenção: O arquivo de estado (terraform.tfstate) contém informações sensíveis como IDs de recursos e, em alguns casos, valores de variáveis. Nunca commit ele no

```
[hel]
terraform {
  backend "s3" {
    bucket        = "meu-terraform-state-bucket"
    key           = "infra/web-server/terraform.tfstate"
    region        = "us-east-1"
    encrypt       = true
    dynamodb_table = "terraform-state-lock"
  }
}
```

Boas práticas de state management:

- **Nunca edite o state manualmente** -- use `terraform state` commands se precisar
- **Use backend remoto** (S3, Terraform Cloud, Azure Storage) -- state local é perda de dados esperando acontecer
- **Ative bloqueio (locking)** -- DynamoDB para S3, ou nativo no Terraform Cloud
- **Separe ambientes** -- um state para dev, outro para staging, outro para production
- **Versãoamento do bucket de state** -- habilite versãoamento no S3 para recuperar states corrompidos

IaC e Versionamento

```
[bash]
infra/
├── environments/
│   ├── dev/
│   │   └── main.tf
│   └── staging/
```

| | └─ main.tf

| └─ production/

| └─ main.tf

|— modules/

| └─ web-server/

| | — main.tf

| | |— variables.tf

| | └─ outputs.tf

| database/

| — main.tf

| — variables.tf

| └─ outputs.tf

└─ global/

└ iam.tf

Dica: Siga o principio DRY (Don't Repeat Yourself) com modulos Terraform. Crie modulos reutilizaveis para web server, database, load balancer -- e reutilize entre

Dica: Siga o princípio DRY (Don't Repeat Yourself) com modulos Terraform. Crie modulos reutilizaveis para web server, database, load balancer -- e reutilize entre ambientes mudando apenas as variaveis.

2. Pipeline Visual com Mermaid

Um pipeline de CI/CD é a espinha dorsal da entrega de software. Ele

[Diagrama] Mermaid

```
graph LR
    subgraph DEV["Desenvolvimento"]
        A["fa:fa-code Commit"] --> B["fa:fa-code-branch Pull Request"]
    end
    subgraph CI["Integracao Continua"]
        C["fa:fa-wrench Build"] --> D["fa:fa-vial Testes Unitarios"]
        D --> E["fa:fa-vials Testes de Integracao"]
        E --> F["fa:fa-shield Seguranca SAST"]
        F --> G["fa:fa-weight-hanging Analise de Qualidade"]
    end
    subgraph CD["Entrega Continua"]
        H["fa:fa-box Build da Imagem Docker"] --> I["fa:fa-registered Push Registry"]
        I --> J["fa:fa-rocket Deploy Staging"]
        J --> K["fa:fa-check-circle Testes E2E"]
    end
    subgraph DEPLOY["Deploy Producao"]
        L["fa:fa-gate Aprovacao Manual"] --> M["fa:fa-chart-line Canary 10%"]
        M --> N["fa:fa-chart-line Canary 50%"]
        N --> O["fa:fa-chart-line Canary 100%"]
        O --> P["fa:fa-medal Producao"]
    end
    subgraph MONITOR["Monitoramento"]
        Q["fa:fa-eye Health Checks"] --> R["fa:fa-bell Alertas"]
        R --> S["fa:fa-rotate-left Rollback Automatico"]
    end
    B --> C
    G --> H
    K --> L
    P --> Q
    style DEV fill:#E3F2FD,stroke:#1A237E
    style CI fill:#F3E5F5,stroke:#7B1FA2
```

style CD fill:#E8F5E9,stroke:#2E7D32

```
style DEPLOY fill:#FFF8E1,stroke:#F57F17
```

```
style MONITOR fill:#FFEBEE,stroke:#C62828
```

Explicação passo a passo

1. Desenvolvimento: Tudo começa com um *commit* e um *Pull*

Request

2. Integração Contínua (CI): Build -> Testes unitários -> Testes de integração -> Segurança SAST -> Análise de qualidade. Se

qualquer etapa falhar, o pipeline para

3. Entrega Contínua (CD): Build da imagem Docker -> Push no registry -> Deploy em staging -> Testes E2E

4. Deploy em Produção: Aprovação manual -> Canary 10% -> 50%

5. Monitoramento: Health checks -> Alertas -> Rollback

Dica: Otimize seu pipeline com execucao paralela: rode testes unitarios e analise de qualidade ao mesmo tempo. Use *matrix* no GitHub Actions ou *parallel* no GitLab CI.

Um pipeline de 15 minutos cai para 6 minutos com paralelismo bem planejado.

3. Estratégias de Deploy

Blue-Green Deployment

Duas versões completas do ambiente coexistem. O balanceador de

[Diagrama] Mermaid

```
graph LR
    USERS["fa:fa-users Usuarios"] --> LB["fa:fa-scale-balanced Load Balancer"]
    subgraph BLUE["Blue (v1.0 - Atual)"]
        B1["App Blue"] --> B2["DB Blue"]
    end
    subgraph GREEN["Green (v1.1 - Nova)"]
        G1["App Green"] --> G2["DB Green"]
    end
    LB -->|"Trafego Atual"| BLUE
    LB -.->|"Switch: DNS / ALB"| GREEN
    GREEN --> HEALTH["Health Check<br/>Ok?"]
    HEALTH -->|"Sim"| CUTOVER["Cutover: 100%<br/>para Green"]
    HEALTH -->|"Nao"| ROLLBACK_G["Rollback:<br/>volta para Blue"]
    style USERS fill:#E3F2FD,stroke:#1A237E
    style LB fill:#F3E5F5,stroke:#7B1FA2
    style BLUE fill:#E3F2FD,stroke:#1565C0
```

```
style GREEN fill:#E8F5E9,stroke:#2E7D32
```


style CUTOVER fill:#E8F5E9,stroke:#2E7D32

```
style ROLLBACK_G fill:#FFEBEE,stroke:#C62828
```

Vantagens: Rollback instantâneo, zero downtime, ambiente Green testável antes do switch.

Desvantagens: Custo dobrado de infraestrutura.

Quando usar: aplicações críticas onde rollback rápido é essencial.

Canary Deployment

Uma fração do tráfego é direcionada para a nova versão, aumentando

[Diagrama] Mermaid

```
graph LR
    LB_C["fa:fa-scale-balanced Load Balancer"] -->|"90%"| STABLE["v1.0 (stable)"]
    LB_C -->|"10%"| CANARY["v1.1 (canary)"]
    CANARY --> MON_C["Monitoramento<br/>5 min"]
    MON_C -->|"OK: Erro < 1%"| STEP2["v1.1 50%"]
    MON_C -->|"FAIL: Erro > 5%"| ABORT["ABORTAR<br/>volta para v1.0"]
    STEP2 --> MON_C2["Monitoramento<br/>5 min"]
    MON_C2 -->|"OK"| FULL["v1.1 100%"]
    MON_C2 -->|"FAIL"| ABORT
    style STABLE fill:#E3F2FD,stroke:#1565C0
    style CANARY fill:#FFF8E1,stroke:#F57F17
    style ABORT fill:#FFEBEE,stroke:#C62828
    style FULL fill:#E8F5E9,stroke:#2E7D32
```

Vantagens: Risco reduzido, rollback natural, dados reais de performance.

Desvantagens: Requer monitoramento em tempo real.

Quando usar: rollouts que exigem validação com tráfego real.

Rolling Update

Substitui instâncias uma a uma (ou em lotes), sem criar um ambiente

[yaml]

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app-deployment
spec:
  replicas: 5
```

strategy:

type: RollingUpdate

rollingUpdate:

maxSurge: 1

maxUnavailable: 0

template:

spec:

containers:

- name: app

```
image: meuregistro/app:1.2.0
```

```
readinessProbe:
```

httpGet:

```
path: /health
```



```
port: 3000
```

```
initialDelaySeconds: 5
```

```
periodSeconds: 10
```

Vantagens: Sem custo extra de infra, zero downtime com `maxUnavailable: 0`.

Desvantagens: Rollback mais lento, versões coexistem.

Quando usar: deploys rotineiros em Kubernetes.

Feature Flags

Feature flags ativam/desativam funcionalidades em produção sem

```
[typescript]
import { getContext, isEnabled } from '@unleash/nextjs';

export default async function Page() {
  const ctx = getContext();
  const novoCheckout = await isEnabled('checkout-v2', ctx);
  if (novoCheckout) {
    return <CheckoutV2 />;
  }
  return <CheckoutV1 />;
}
```

Ferramentas: LaunchDarkly, Unleash, Flagsmith, GrowthBook,

Dica: Não use feature flags como desculpa para não ter uma boa estratégia de deploy.

Feature flags complementam, não substituem, estratégias como Blue-Green e Canary.

4. Banco de Dados e Migrações

Prisma Migrate Workflow

```
[prisma]
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

```
model User {
```

```
id      String    @id @default(cuid())
```

email String @unique

name String?

role	Role	@default(VIEWER)
------	------	------------------

```
createdAt DateTime @default(now())
```

updatedAt DateTime @updatedAt

}


```
enum Role {
```

ADMIN

EDITOR

VIEWER

}

[bash]

```
npx prisma migrate dev --name add-project-model
```

```
npx prisma migrate deploy
```

```
npx prisma migrate status
```

[Diagrama] Mermaid

graph TD


```
SCHEMA["Editar schema.prisma"] --> GEN_MIG["npx prisma migrate dev<br/>--name descri
```

```
GEN_MIG --> SQL_GEN["Gera arquivo SQL<br/>em prisma/migrations/"]
```

```
SQL_GEN --> REVIEW["Revisar SQL gerado"]
```

```
REVIEW -->|"SQL correto"| COMMIT_MIG["Commit no Git"]
```

```
REVIEW -->|"SQL incorreto"| FIX_MIG["Corrigir SQL<br/>manualmente"]
```

FIX_MIG --> REVIEW

```
COMMIT_MIG --> DEPLOY_MIG["CI/CD executa:<br/>prisma migrate deploy"]
```

```
DEPLOY_MIG -->|"Sucesso"| DONE["Schema atualizado"]
```



```
DEPLOY_MIG -->|"Falha"| ROLLBACK_MIG["Estrategia de rollback"]
```

`style SCHEMA fill:#E3F2FD,stroke:#1A237E`

```
style GEN_MIG fill:#F3E5F5,stroke:#7B1FA2
```

```
style SQL_GEN fill:#FFF8E1,stroke:#F57F17
```

`style REVIEW fill:#E3F2FD,stroke:#1A237E`

`style DONE fill:#E8F5E9,stroke:#2E7D32`

```
style ROLLBACK_MIG fill:#FFEBEE,stroke:#C62828
```

Estrategia de Rollback de Migracao

Atencao: Backup antes de migration nao e opcional -- e obrigatorio. Ja vi uma migration que parecia segura travar o banco por 40 minutos porque a tabela tinha 50

```
[bash]
#!/bin/bash
set -euo pipefail
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
BACKUP_FILE="/tmp/pre_migration_${TIMESTAMP}.sql"
pg_dump --no-owner --no-acl --format=custom --file="${BACKUP_FILE}" "${DATABASES}
```

Fluxo de Rollback Ilustrado

[Diagrama] Mermaid

```
graph TD
    START["Inicio da Migracao"] --> BACKUP_DB["pg_dump backup"]
    BACKUP_DB --> RUN_MIG["Executar migration"]
    RUN_MIG -->|"Sucesso"| VERIFY["Verificar integridade"]
    VERIFY -->|"OK"| SUCCESS["Migration concluida"]
    VERIFY -->|"Inconsistencia"| TRIGGER_RB["Gatilho de rollback"]
    RUN_MIG -->|"Erro"| TRIGGER_RB
    TRIGGER_RB --> RESTORE["Restaurar backup"]
    RESTORE --> ALERT["Notificar time"]
    style SUCCESS fill:#E8F5E9,stroke:#2E7D32
    style TRIGGER_RB fill:#FFEBEE,stroke:#C62828
    style RESTORE fill:#FFEBEE,stroke:#C62828
```

5. Exercicio Pratico

Título: Escolha a estrategia de deploy certa

Duracao: 5 minutos

Cenário 1

Mudança de layout no checkout. Frontend puro, backend inalterado.

Time quer validar com usuários reais antes de liberar para todos.

- [] a) Blue-Green Deployment

- ☐ b) Canary Deployment
- ☐ c) Rolling Update
- ☐ d) Feature Flags

Cenário 2

Migração de monólito para microsserviços. 50 mil usuários ativos.

Rollback precisa ser instantâneo

- ☐ a) Blue-Green Deployment
- ☐ b) Canary Deployment
- ☐ c) Rolling Update
- ☐ d) Feature Flags

Cenário 3

Cluster Kubernetes com 20 microsserviços. Serviço interno sem exposição externa. Zero downtime sem custo extra.

- ☐ a) Blue-Green Deployment
- ☐ b) Canary Deployment
- ☐ c) Rolling Update
- ☐ d) Feature Flags

Cenário 4

Novo algoritmo de recomendação. Validação offline ok, mas quer testar impacto em métricas de negócio.

- ☐ a) Blue-Green Deployment
- ☐ b) Canary Deployment
- ☐ c) Rolling Update
- ☐ d) Feature Flags

Gabarito

Cenário	Resposta	Justificativa
---------	----------	---------------

1

d) Feature Flags

Mudança de frontend,
testes A/B, kill switch
instantâneo

a) Blue-Green

Rollback instantâneo, ambientes paralelos
--

c) Rolling Update

Padrão Kubernetes, sem
custo extra

b) Canary Deployment

Validação com tráfego real,
progressão gradual

Recapitulando: Neste capítulo você aprendeu os fundamentos de Infrastructure as Code com Terraform; a estrutura completa de um pipeline de CI/CD com diagrama visual; as estratégias de deploy Blue-Green, Canary, Rolling Update e Feature Flags com seus casos de uso; e o workflow seguro de migrações de banco de dados com

Recapitulando: Neste capítulo você aprendeu os fundamentos de Infrastructure as Code com Terraform; a estrutura completa de um pipeline de CI/CD com diagrama visual; as estratégias de deploy Blue-Green, Canary, Rolling Update e Feature Flags com seus casos de uso; e o workflow seguro de migrações de banco de dados com Prisma Migrate, backup obrigatório e rollback automatizado.

Seguranca, Monitoramento e Self-Healing

1. Seguranca no Pipeline (SAST/DAST)

A seguranca automatizada se organiza em **três camadas** que acompanham o ciclo de vida do artefato: análise estática no código,

Definicao: SAST (Static Application Security Testing) analisa o codigo-fonte sem executa-lo. DAST (Dynamic Application Security Testing) testa a aplicacao em

[Diagrama] Mermaid

```
graph LR
    CODE["Codigo<br/>SAST"] --> BUILD_IMG["Imagem<br/>Trivy"]
    BUILD_IMG --> RUNTIME["Runtime<br/>DAST"]
    CODE -->| "SonarQube<br/>CodeQL<br/>ESLint" | CODE_RES["OK ou Falha"]
    BUILD_IMG -->| "Trivy<br/>Docker Scan" | IMG_RES["OK ou Falha"]
    RUNTIME -->| "OWASP ZAP" | RUNTIME_RES["OK ou Falha"]
    CODE_RES -->| "Falha" | FIX["Corrigir"]
    IMG_RES -->| "Falha" | FIX
    RUNTIME_RES -->| "Falha" | FIX
    FIX --> CODE
    style CODE fill:#E3F2FD,stroke:#1A237E
    style BUILD_IMG fill:#FFF8E1,stroke:#F57F17
    style RUNTIME fill:#F3E5F5,stroke:#7B1FA2
    style FIX fill:#FFEBEE,stroke:#C62828
```

SAST -- Analise Estatica

Ferramentas de SAST escaneiam o código em busca de padrões inseguros: SQL injection, XSS, vazamento de secrets, uso de

[yaml]

```
sast-analysis:
```


runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: ESLint Security

```
run: npx eslint . --ext .ts,.tsx --config .eslintrc-security.js
```

- name: CodeQL

```
uses: github/codeql-action/init@v3
```

with:


```
languages: javascript-typescript
```

queries: security-extended,security-and-quality

- name: CodeQL Analyze

uses: github/codeql-action/analyze@v3

- name: SonarQube Scan

uses: sonarsource/sonarcloud-github-action@master

Dica: CodeQL tem duas categorias: security-extended (recomendado) e security-and-quality (mais abrangente, mas mais falsos positivos). Comece com security-extended

Dica: CodeQL tem duas categorias: security-extended (recomendado) e security-and-quality (mais abrangente, mas mais falsos positivos). Comece com security-extended

Ferramenta	Tipo	Cobertura	Custo
SonarQube	Plataforma	Qualidade + Segurança	Gratuito / Pago
CodeQL	Motor semântico	Segurança profunda	Gratuito para públicos
ESLint + plugins	Linters	Padrões OWASP Top 10	Gratuito
Semgrep	Pattern matching	Regras customizáveis	Gratuito / Pago
Snyk Code	IA + SAST	Multilinguagem	Pago

DAST -- Análise Dinâmica

DAST testa a aplicação **em execução**, simulando ataques reais

```
[yaml]
dast-analysis:
  needs: deploy-staging
  runs-on: ubuntu-latest
  steps:
    - name: OWASP ZAP Full Scan
      uses: zaproxy/action-full-scan@v0.12
      with:
        target: https://staging.meusistema.com.br
        fail_action: true
    - name: Publicar relatório
      uses: actions/upload-artifact@v4
      with:
        name: zap-report
        path: report_html/
```

Atenção: DAST em produção exige cuidado extremo. Testes como SQL injection podem degradar ou derrubar o serviço. Execute DAST sempre em staging.

Dependency Scanning com IA

Dependabot e Renovate monitoram dependências e abrem PRs


```
[yaml]
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "daily"
    open-pull-requests-limit: 15
    labels:
      - "dependencies"
      - "security"
    reviewers:
      - "security-squad"
```

Trivy varre containers, sistemas de arquivos e repositórios Git em

```
[yaml]
container-scan:
  runs-on: ubuntu-latest
  steps:
    - name: Build image
      run: docker build -t app:${{ github.sha }} .
    - name: Trivy Scan
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: app:${{ github.sha }}
        severity: CRITICAL,HIGH
        exit-code: 1
```

Definicao: CVE (Common Vulnerabilities and Exposures) e um identificador publico de vulnerabilidade de seguranca. Quando uma CVE e publicada para uma biblioteca que voce usa, Dependabot ou Renovate abrem um PR automaticamente.

2. Code Review Automatizado

Automated PR Checks

Antes de qualquer humano revisar, uma bateria de verificações

```
[yaml]
```

pr-checks:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

with:

fetch-depth: 0

```
- run: pnpm lint
```


- run: pnpm format:check

- run: pnpm typecheck

```
- run: pnpm test:unit
```

```
- run: pnpm test:integration
```

```
- run: npm audit --audit-level=high
```

- name: Check PR size

```
uses: actions/github-script@v7
```

with:

script: |

```
const pr = await github.rest.pulls.get({ ...context.repo, pull_numbe
```

```
if (pr.data.changed_files > 20) {
```

```
core.warning('PR grande: ' + pr.data.changed_files + ' arquivos')
```

}

AI Code Review Bots

Ferramenta	Modelo	Recursos
CodeRabbit	GPT-4 / Claude	Revisão por chat, sugestões inline
OpenCode Code Review	Multi-model	
		Regras do time, integração nativa
GitHub Copilot Code Review	OpenAI	Revisão direta no GitHub

Bots de revisao nao substituem revisao humana -- eles sao um *first pass*. Times que usam AI code review detectam 30% mais bugs em PRs grandes.

CODEOWNERS

```
[yaml]
# .github/CODEOWNERS
* @time-core
/packages/payment/ @time-financeiro
/.github/workflows/security-* @ciso
/terraform/ @time-sre
/docs/ @tech-writer
```

3. Releases e Versionamento

Definicao: Semantic Versioning (SemVer) e o padrao de versionamento no formato MAJOR.MINOR.PATCH. MAJOR: mudancas que quebram compatibilidade. MINOR: novas funcionalidades retrocompativeis. PATCH: correcoes de bugs

```
[markdown]
v1.4.0 -> v2.0.0 (MAJOR: quebra compatibilidade)
v1.4.0 -> v1.5.0 (MINOR: nova feature)
v1.4.0 -> v1.4.1 (PATCH: correcao de bug)
```

Conventional Commits

```
feat: adiciona suporte a PIX no checkout      -> MINOR
fix: corrige validacao de CPF                  -> PATCH
```

862

BREAKING CHANGE: migra para nova API -> MAJOR

chore: atualiza dependencias

-> Sem release

docs: atualiza README

-> Sem release

semantic-release

```
[yaml]
name: Release Automatica
on:
  push:
    branches: [main]
jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: pnpm/action-setup@v3
      - uses: actions/setup-node@v4
      - run: pnpm install --frozen-lockfile
      - name: Semantic Release
        uses: cycjimmy/semantic-release-action@v4
        with:
          extra_plugins: |
            @semantic-release/changelog
            @semantic-release/git
```

Dica: O CHANGELOG gerado automaticamente e a melhor documentacao viva que existe. Ele reflete exatamente o que foi entregue, sem depender de memoria humana.

4. Ambientes Efemeros

Definicao: Ambiente efemero (*ephemeral environment*) e uma replica completa do ambiente de producao (servico + banco + dependencias) criada automaticamente

```
[yaml]
deploy-preview:
  if: github.event_name == 'pull_request'
  runs-on: ubuntu-latest
  environment:
    name: pr-${{ github.event.number }}
    url: https://pr-${{ github.event.number }}.app.meusistema.com
  steps:
```

- uses: actions/checkout@v4

```
- run: pnpm install --frozen-lockfile
```

- run: pnpm build

- name: Deploy no ECS

run: |

```
aws ecs update-service \
```



```
--cluster preview-cluster \
```

```
--service app-preview-${{ github.event.number }} \
```

--force-new-deployment

Plataforma

Tipo

Cleanup automático

Vercel Preview

Serverless

Sim

Render

PaaS

Sim

ECS + Fargate

Container

Manual (via workflow)

Railway

PaaS

Sim

Atencao: Ambiente efemero esquecido custa dinheiro. Uma database db.t3.micro

Atencao: Ambiente efemero esquecido custa dinheiro. Uma database db.t3.micro

```
[yaml]
cleanup-stale-environments:
  runs-on: ubuntu-latest
  schedule:
    - cron: "0 6 * * *"
  steps:
    - name: Listar PRs abertos
      run: |
        open_prs=$(gh pr list --state open --json number --jq '.[].number')
    - name: Identificar ambientes orfaos
      run: |
        for service in $(aws ecs list-services --cluster preview-cluster --que
        pr_number=$(echo $service | grep -oP 'preview-\K\d+')
        if [[ ! " $open_prs " =~ " $pr_number " ]]; then
          aws ecs delete-service --cluster preview-cluster --service $service
        fi
        done
```

5. Monitoramento e Self-Healing

Observabilidade: Metricas, Logs e Tracing

Ferramenta	Métricas	Logs	Tracing	Custo
Datadog	Sim	Sim	Sim	\$\$\$
New Relic	Sim	Sim	Sim	\$\$\$
Grafana + Prometheus	Sim	Via Loki	Via Tempo	Gratuito
SigNoz	Sim	Sim	Sim	Open source

Health Checks e Readiness Probes

```
[yaml]
apiVersion: apps/v1
kind: Deployment
```

metadata:

name: app

spec:

replicas: 3

template:

spec:

containers:

- name: app

image: meuregistro/app:1.5.0

livenessProbe:

httpGet:

path: /healthz

port: 3000

```
initialDelaySeconds: 10
```

```
periodSeconds: 15
```

readinessProbe:

httpGet:

path: /readyz

port: 3000

```
initialDelaySeconds: 5
```



```
periodSeconds: 10
```

[typescript]

```
server.get('/healthz', async (_req, reply) => {
```

```
reply.status(200).send({ status: 'ok' })
```

})


```
server.get('/readyz', async (_req, reply) => {
```

```
const dbHealthy = await checkDatabaseConnection()
```



```
if (!dbHealthy) {
```

```
reply.status(503).send({ status: 'not ready' })
```

return

}

```
reply.status(200).send({ status: 'ready' })
```

})

Self-Healing com IA

Self-healing é a capacidade do sistema de detectar uma anomalia e co

[Diagrama] Mermaid

graph TB

```
METRICA["Metrica anomala<br/>P99 > 500ms"] --> ALERTA["Alerta disparado"]
ALERTA --> DECISAO{"Diagnostico<br/>automatico"}
DECISAO -->|"Container crashando"| REINICIAR["Reiniciar container"]
DECISAO -->|"Trafego excessivo"| ESCALAR["Escalar horizontalmente"]
DECISAO -->|"Bug no deploy"| ROLLBACK["Rollback automatico"]
REINICIAR --> VERIFICAR["Verificar health check"]
ESCALAR --> VERIFICAR
ROLLBACK --> VERIFICAR
VERIFICAR -->|"ok"| OK["Sistema saudavel"]
VERIFICAR -->|"falha"| ESCALAR_HUMANO["Escalar para SRE"]
style METRICA fill:#FFF8E1,stroke:#F57F17
style ALERTA fill:#FFEB3B,stroke:#C62828
style DECISAO fill:#F3E5F5,stroke:#7B1FA2
style OK fill:#E8F5E9,stroke:#2E7D32
style ESCALAR_HUMANO fill:#FFEB3B,stroke:#C62828
```

Atencao: Self-healing nao e magica. Comece com cenarios simples: reiniciar container que crashou, escalar horizontalmente sob carga, fazer rollback de deploy com erro.

Runbook Automatizado

[yaml]

```
name: Runbook - Recuperar Database
on:
  workflow_dispatch:
    inputs:
      database_name:
        description: "Nome do banco a recuperar"
        required: true
jobs:
  restore:
    runs-on: ubuntu-latest
    environment: production
    steps:
```

- name: Parar aplicacao


```
run: kubectl scale deployment app --replicas=0 -n production
```

- name: Restaurar snapshot

```
run: echo "Restaurando ultimo snapshot..."
```

- name: Subir aplicacao

```
run: kubectl scale deployment app --replicas=3 -n production
```

- name: Notificar time

```
run: curl -X POST ${ secrets.SLACK_WEBHOOK }
```

Recapitulando: Self-healing reduz o MTTR (Mean Time To Recovery) de minutos para segundos em cenários comuns: container crashando, pico de trafego, deploy com

Recapitulando: Self-healing reduz o MTTR (Mean Time To Recovery) de minutos para segundos em cenários comuns: container crashando, pico de tráfego, deploy com bug. Comece com runbooks simples, automatize um por um.

6. Exercício Prático

Título: Design de Pipeline de Segurança para Microserviço
Cenário: Microserviço em Node.js/TypeScript em Kubernetes. Time de 6 devs, deploys múltiplos ao dia, GitHub Actions.
Tarefa: Projete uma pipeline de segurança completa cobrindo as três camadas (código -> imagem -> runtime) e inclua self-healing.
Checklist de Verificação: Configurado com CodeQL (queries `security-extended`)

- ☐ ESLint com plugins de segurança
- ☐ SonarQube configurado
- ☐ Dependabot ativo com revisores designados
- ☐ Trivy escaneando imagem com `exit-code: 1`
- ☐ OWASP ZAP rodando contra staging
- ☐ CODEOWNERS com time de segurança
- ☐ Semantic release com Conventional Commits
- ☐ Ambiente efêmero criado no PR e destruído no merge
- ☐ Cron job de limpeza de ambientes órfãos
- ☐ Health checks (liveness + readiness)
- ☐ Self-healing runbook para reinicialização de container

Resumo do Capítulo

Conceito	Ferramentas	Objetivo
SAST	SonarQube, CodeQL, ESLint	Análise estática de segurança no código
DAST		Teste dinâmico simulando ataques reais
Dependency Scan	Dependabot, Renovate,	

Trivy

Varredura de CVEs em

dependências

Code Review Auto

CodeRabbit,
CODEOWNERS

Primeira camada de revisão com IA

Versionamento

semantic-release,
Conventional Commits

Releases e CHANGELOG
automáticos

Ambientes Efêmeros

Vercel, Render, ECS

Preview por branch,
destruição automática

Observabilidade

Datadog, Prometheus	Grafana,
------------------------	----------

Métricas, logs e tracing

Health Checks

Liveness, Readiness

Sondas Kubernetes

Self-Healing

Runbooks automáticos

Detecção e correção sem humano

Cultura de Automacao e Proximos Passos

1. O Custo da Automacao

Automação não é de graça. Toda pipeline que você cria exige tempo para ser construída, mantida, debugada e evoluída.

A Regra das 3 Execucoes

Dica: Automatize apenas depois de executar a mesma tarefa manualmente 3 vezes.

- Nem antes, nem depois.
1. ****1 execucao****: você ainda não entendeu o processo direito. Automatizar agora é especular.
 2. ****2 execucoes****: você começa a enxergar padrões, mas ainda pode mudar o processo.
 3. ****3 execucoes****: o processo está maduro. Você sabe exatamente o que precisa ser automatizado.

Atencao: A Regra das 3 Execucoes tem uma excecao: tarefas de segurancra critica devem ser automatizadas na primeira ocorrencia.

Analise de Custo vs Beneficio

Custo de automatizar = (horas para criar pipeline + horas/ano de manutencao) x

Beneficio anual = (horas economizadas por execucao x execucoes/ano) x custo-ho

ROI = (beneficio anual - custo) / custo x 100

Cenário	Custo criar	Manutenção/ano	ROI 1º ano
Deploy manual -> pipeline	8h	4h	1.150%
Lint manual -> automático	2h	1h	1.233%
Teste manual -> CI gate	16h	8h	450%

Dica: Trate suas pipelines como trata seu codigo de aplicacao: code review, testes,



Dica: Trate suas pipelines como trata seu código de aplicação: code review, testes, documentação e refatoração periódica.

2. Pipeline as Code Reutilizavel

Design Modular de Pipelines

```
pipeline/
├── modules/
│   ├── build.yaml
│   ├── test.yaml
│   ├── lint.yaml
│   ├── security.yaml
│   ├── deploy.yaml
│   └── notify.yaml
├── workflows/
│   ├── ci.yaml
│   └── cd.yaml
└── templates/
    └── deploy-template.yaml
```

Workflows Reutilizaveis no GitHub Actions

```
[yaml]
name: Deploy Template
on:
  workflow_call:
    inputs:
      environment:
        required: true
        type: string
      image-tag:
        required: true
        type: string
    secrets:
      CLOUD_API_KEY:
        required: true
```


jobs:

deploy:

```
runs-on: ubuntu-latest
```

```
environment: ${ inputs.environment }
```

steps:

- name: Autenticar

```
run: echo "${{ secrets.CLOUD_API_KEY }}" | docker login --password-std
```

- name: Pull imagem


```
run: docker pull meuregistro/app:${{ inputs.image-tag }}
```

- name: Health check

```
run: curl -f http://localhost:3000/health || exit 1
```

```
[yaml]
jobs:
  deploy:
    uses: org/deploy-templates/.github/workflows/deploy-template.yaml@v1
    with:
      environment: production
      image-tag: ${ github.sha }
    secrets:
      CLOUD_API_KEY: ${ secrets.CLOUD_API_KEY }
```

Definição: Internal Developer Platform (IDP) é o conjunto de ferramentas, templates e padrões que um time de plataforma oferece para que os times de produto entreguem software com autonomia.

3. Boas Práticas Consolidadas

CI/CD

#	Prática	Capítulo	Status
1	Pipeline como código versionado	Cap 3	
2	Testes automáticos como gate	Cap 3	
3	Cache inteligente de dependências	Cap 3	
4	Workflows reutilizáveis	Cap 7	

Deploy

#	Prática	Capítulo	Status
5	Estratégia de deploy definida	Cap 4	
6	Health check automatizado	Cap 4	
7	Rollback testado	Cap 4	

Database

#	Pratica	Capitulo	Status
8	Migration como parte do deploy	Cap 4	
9	Backup antes de migration	Cap 4	
10	Testes estruturais para infraestrutura de dados	Cap 5	

Seguranca

#	Pratica	Capitulo	Status
11	Secrets nunca no código	Cap 6	
12	SAST + SCA no pipeline	Cap 6	
13	Principio do menor privilégio	Cap 6	

Monitoramento

#	Pratica	Capitulo	Status
14	Alertas baseados em SLOs	Cap 4	
15	Dashboards de DORA metrics	Cap 7	

4. Construindo uma Cultura de Automacao

"Culture eats strategy for breakfast." -- Peter Drucker. Voce pode ter a melhor estratégia de automacao do mundo. Se a cultura do time nao abraçar a mudanca, ela vai fracassar.

Passo a passo pratico:

- ****Identifique a dor****: converse com o time. Qual tarefa manual mais irrita?
- ****Automatize uma tarefa pequena****: algo que leva no máximo 2 dias
- ****Meca o antes e depois****: mostre o tempo economizado com dados reais
- ****Comemore a vitoria****: destaque o ganho no grupo do time
- ****Repita****: escolha a próxima dor e recomece

Metricas que Importam (DORA)

Metrica	O que mede	Time alto desempenho
Deploy Frequency	Frequência de deploys	Múltiplas vezes ao dia
Lead Time for Changes	Commit ao deploy	Menos de 1 hora
Mean Time to Recover	Tempo para recuperar falha	Menos de 1 hora
Change Failure Rate	% deploys que causam incidentes	0-15%

Recapitulando: Como vimos no Capítulo 3, a automacao de CI/CD impacta diretamente as quatro metricas DORA. Uma pipeline bem construida reduz lead time de dias para minutos.

Modelo de Maturidade DevOps

[Diagrama] Mermaid

```
graph LR
  subgraph ESTAGIO1["Manual"]
    M1["Deploys manuais<br/>Sem testes auto"]
  end
  subgraph ESTAGIO2["CI/CD Basico"]
    I1["Pipeline build+teste<br/>Deploy manual"]
  end
  subgraph ESTAGIO3["Automacao Avancada"]
    A1["Deploy automatizado<br/>IaC + Seguranca"]
  end
```

end

```
subgraph ESTAGIO4["Cultura Estabelecida"]
```



```
C1["Multiplos deploys/dia<br/>DORA no dashboard"]
```

end

ESTAGIO1 --> ESTAGIO2 --> ESTAGIO3 --> ESTAGIO4

style ESTAGI01 fill:#FFEBEE,stroke:#C62828

style ESTAGIO2 fill:#FFF8E1,stroke:#F57F17

style ESTAGIO3 fill:#E3F2FD,stroke:#1565C0

style ESTAGI04 fill:#E8F5E9,stroke:#2E7D32

5. Persona: A Jornada de Carlos (Tech Lead)

Carlos é tech lead de um time de 6 desenvolvedores. Quando

começou, o processo de deploy era: `main`

- SSH no servidor de produção
- `git pull && npm install && npm run build`
- Verificação manual
- Torcer para não precisar de rollback
- **Tempo total: 4 horas**

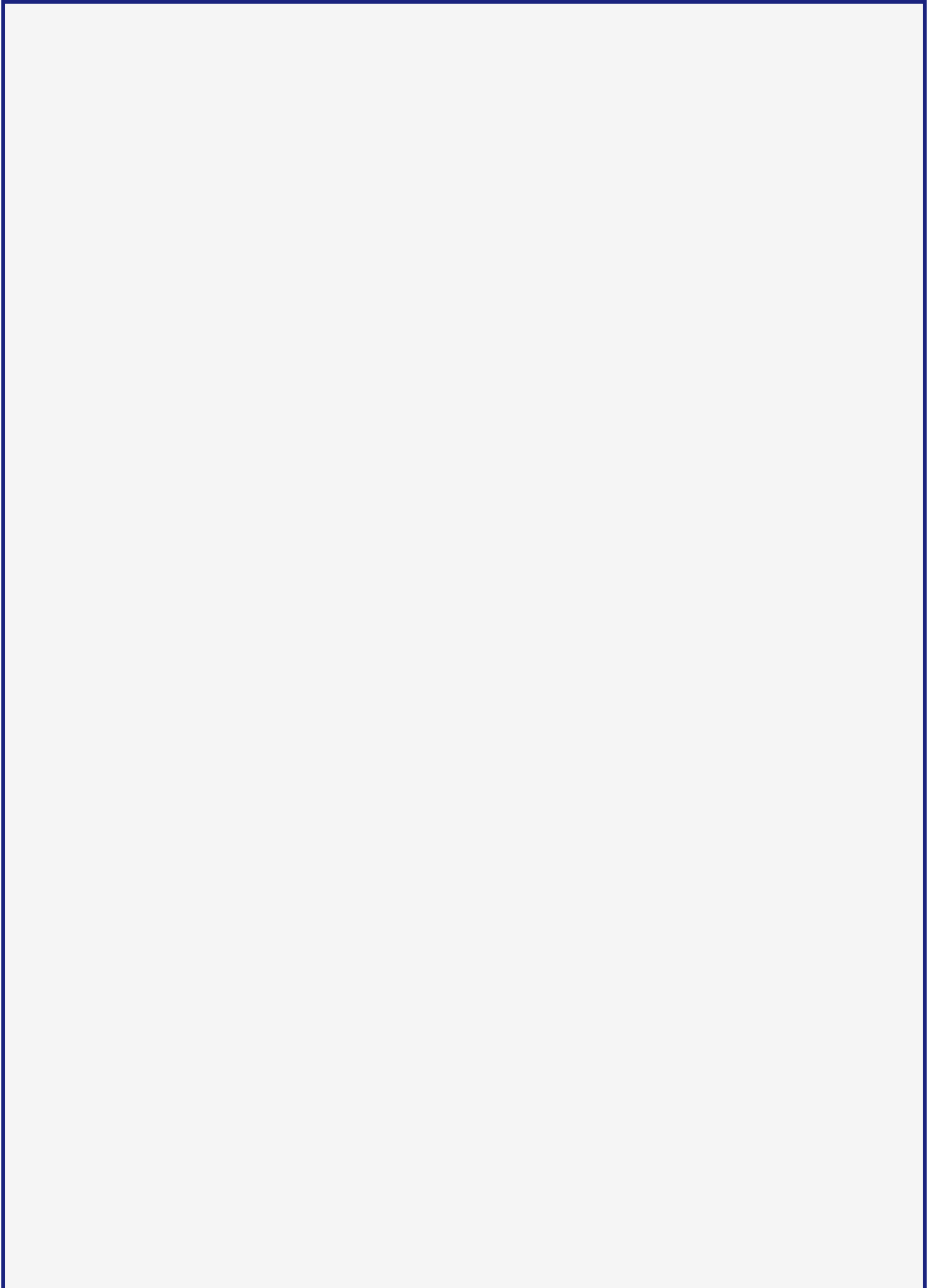
O que Carlos fez

1. **Cap 3 -- CI/CD**: Pipeline GitHub Actions. Lead time caiu de 4h para 30min.
2. **Cap 5 -- IaC**: Migrou para Terraform + Kubernetes. Ambientes declarados como código.
3. **Cap 6 -- Seguranca**: Semgrep + Dependabot. Vulnerabilidade crítica detectada no primeiro scan.
4. **Cap 4 -- Deploy**: Canary com Argo Rollouts. Health check automático.
5. **Cap 7 -- Cultura**: Dashboard DORA compartilhado com o time.

Resultados

Metrica	Antes	Depois	Melhoria
Deploy frequency	1x/semana	3x/dia	15x
Lead time	4 horas	8 minutos	30x
MTTR	2 horas	12 minutos	10x
Change failure rate	40%	8%	5x

Recapitulando: A historia de Carlos nao e ficticia. E o padrao que observei em dezenas de times que implementaram os conceitos deste livro de forma consistente. A



Recapitulando: A historia de Carlos nao e ficticia. E o padrao que observei em dezenas de times que implementaram os conceitos deste livro de forma consistente. A automacao bem feita nao melhora 10% -- melhora 10x.

6. Persona: A Evolucao de Ana (Dev Junior)

Ana é desenvolvedora júnior. Entrou na equipe de Carlos há 6 meses.

Cap 1: Os 5 Erros

Na primeira semana, Ana cometeu todos os 5 erros. Copiou código do Claude sem validar, fez prompts vagos, não iterou.

Cap 2: Agentes

Criou seu primeiro agente: um **Validador deCodigo**. O prompt define regras claras de validação.

Cap 3-4: CI/CD e Deploy

Ajudou Carlos a manter a pipeline. Aprendeu YAML, logs de falha, e fluxo de rollout canary.

Cap 6: Seguranca

Participa das revisões de segurança. Interpreta resultados dos scanners e sugere correções.

Hoje

Ana orienta novos devs juniores. Reconhece cada erro que eles cometem -- porque já cometeu todos.

7. Mapa de Navegacao por Perfil

Perfil	Comece por	Pule	Foque em
Dev Júnior	Cap 1, 2	Cap 5 (parcial)	3, 4, 6
Tech Lead	Cap 3, 4	Cap 1, 2 (revisão)	5, 6, 7
DevOps / SRE	Cap 5, 6	Cap 1	3, 4, 7

Gerente

Cap 7, 1

2, 3

4, 5, 6

8. Mapa do Conhecimento

[Diagrama] Mermaid

```
mindmap
  root((IA para Devs))
    Fundamentos
      Cap 1: Erros Comuns
        Confianca cega
        Prompt vago
        Ignorar seguranca
    Agentes
      Cap 2: Agentes na Pratica
        Anatomia do agente
        Especializacao
    Automacao
      Cap 3: CI/CD com IA
        Pipelines
        Testes
      Cap 4: Deploy e Operacoes
        Blue-Green
        Canary
        Rollback
    Infraestrutura
      Cap 5: IaC
        Terraform
        GitOps
    Seguranca
      Cap 6: Pipeline Seguro
        SAST
        SCA
        Compliance
    Cultura
      Cap 7: Proximos Passos
        Custo da automacao
        Boas praticas
        DORA metrics
        Roadmap 30/60/90
```

9. E Agora? -- Proximos Passos

Roadmap 30/60/90 Dias

Semana	Acao	Capitulo
1	Diagnóstico do pipeline atual	Cap 3
2	Automatize UMA tarefa manual pequena	Cap 3
3	Crie seu primeiro agente especializado	Cap 2
4	Implemente health check no deploy	Cap 4

Semana	Acao	Capitulo
5-6	Migre infraestrutura pequena para IaC	Cap 5
7	Adicione SAST + SCA no pipeline	Cap 6
8	Implemente estrategia de deploy	Cap 4

Semana	Acao	Capitulo
9	Crie dashboard DORA para o time	Cap 7
10	Compartilhe resultados com o time	Cap 7
11	Crie repositorio central de templates	Cap 7
12	Ritual mensal de revisao de automacao	Cap 7

Ferramentas para Explorar

Ferramenta	Para que serve
------------	----------------

OpenCode

Framework de regras de IA para código

Claude Code

Assistente de IA para terminal

GitHub Copilot

Pair programming com IA

Semgrep

SAST scanning open-source

Dependabot

SCA automático

Argo Rollouts

Deploy canary/blue-green no K8s

Terraform

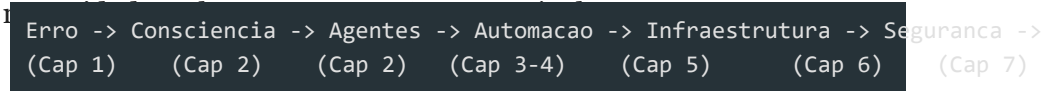
IaC multi-cloud

Pulumi

IaC com TypeScript/Python/Go

10. Conclusão: O Ciclo se Fecha

Começamos com um alerta: 90% dos desenvolvedores cometem os mesmos 5 erros ao usar IA. Terminamos com uma visão de



A IA não vai substituir seu trabalho. Mas um desenvolvedor que sabe usar IA, que automatiza processos, que constrói pipelines seguros e que cultiva uma cultura de automação - esse vai estar muito à frente.

[RECAPITULANDO -- O Legado do Livro]

1. ***IA e ferramenta, não oráculo** -- valide, teste, questione a saída da IA*
2. ***Agentes especializados vencem agentes genéricos** -- estruture seus prompts como agentes*
3. ***Automatize com intenção** -- a Regra das 3 Execuções, análise de custo*
4. ***Pipeline seguro e pipeline completo** -- CI/CD + IaC + Security + Deploy*
5. ***Cultura e o verdadeiro deploy** -- métricas, transparência, melhoria contínua*

Feche o livro.

Abra o terminal e comece.